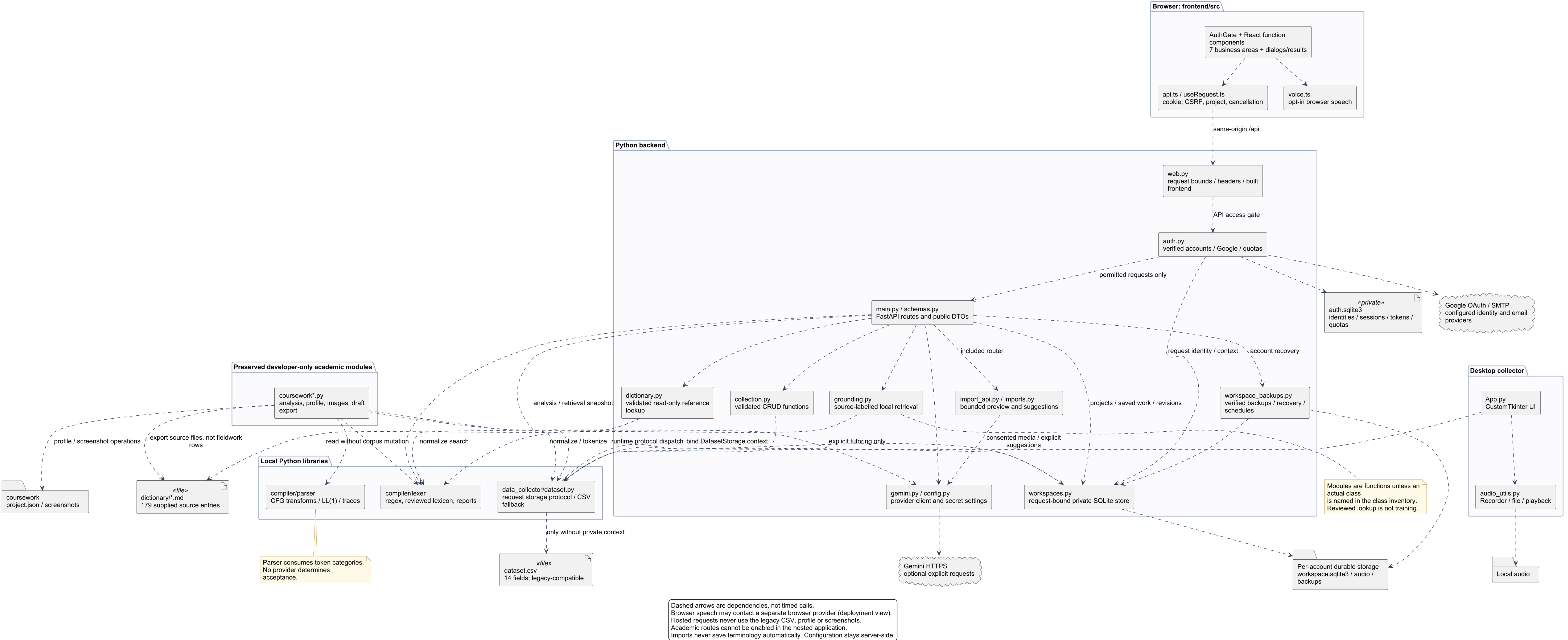
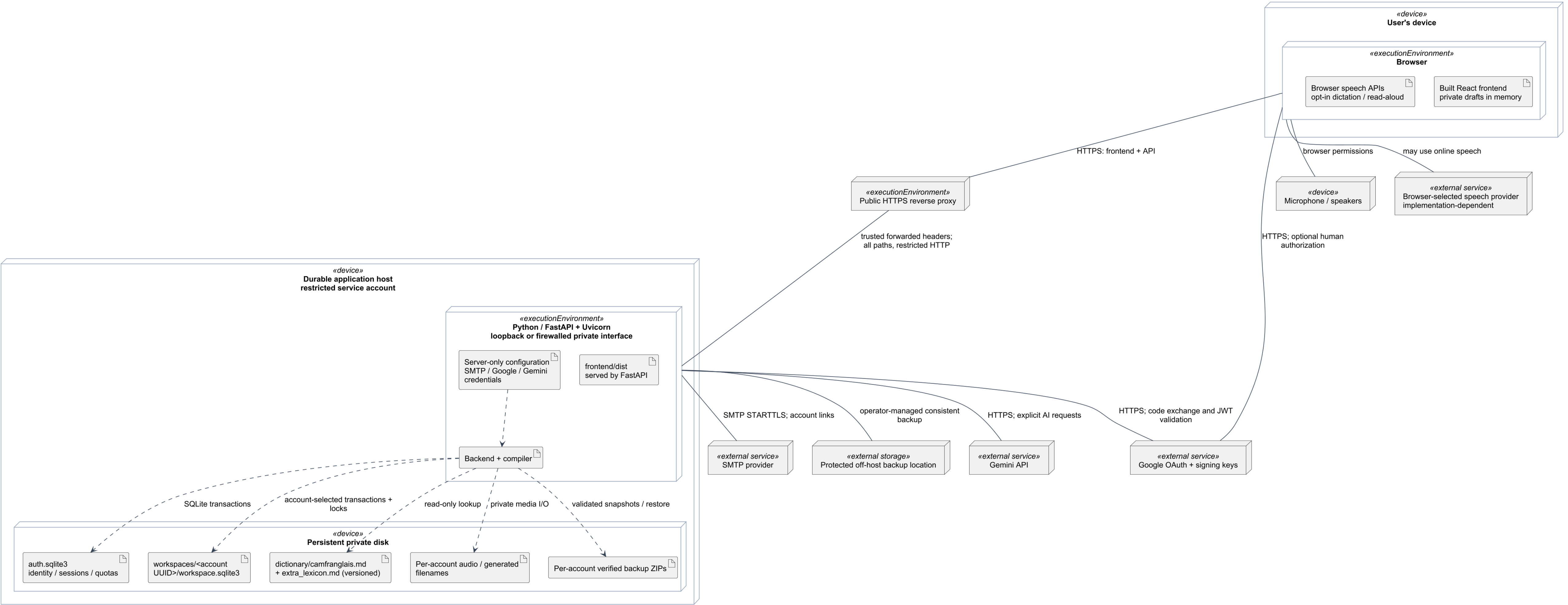


Hosted business components and isolated legacy storage

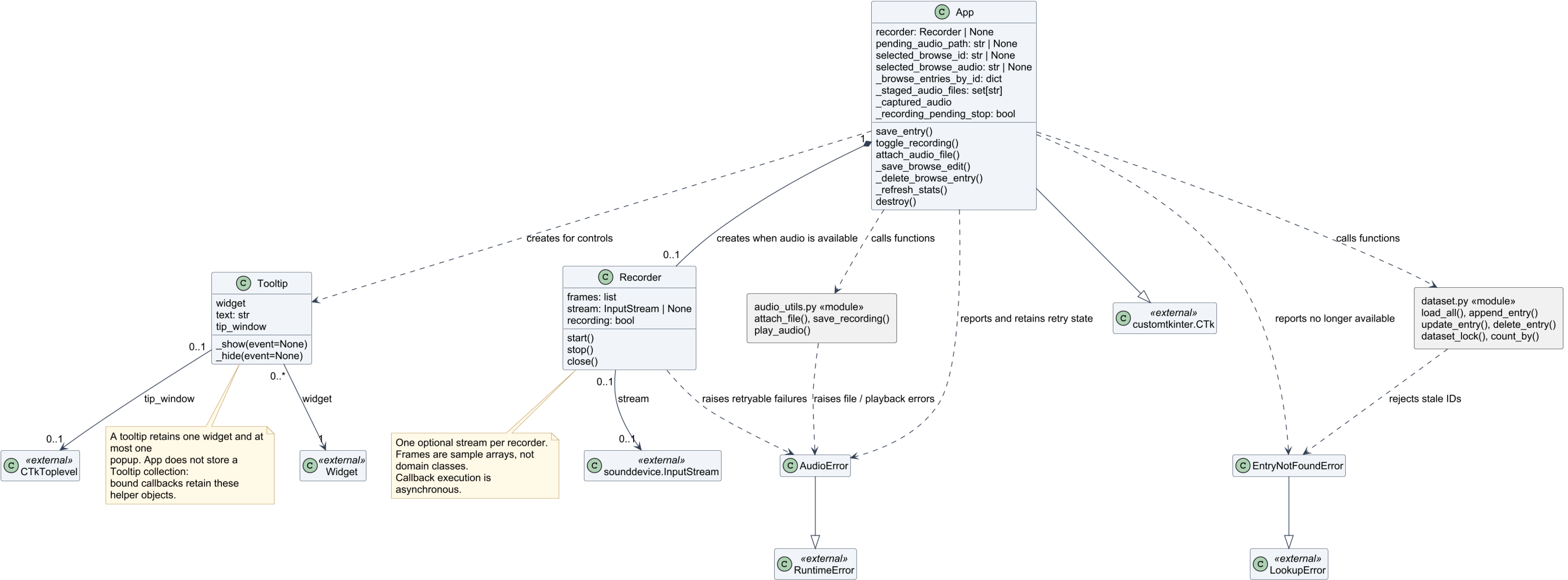


Hosted same-origin application: one durable application host



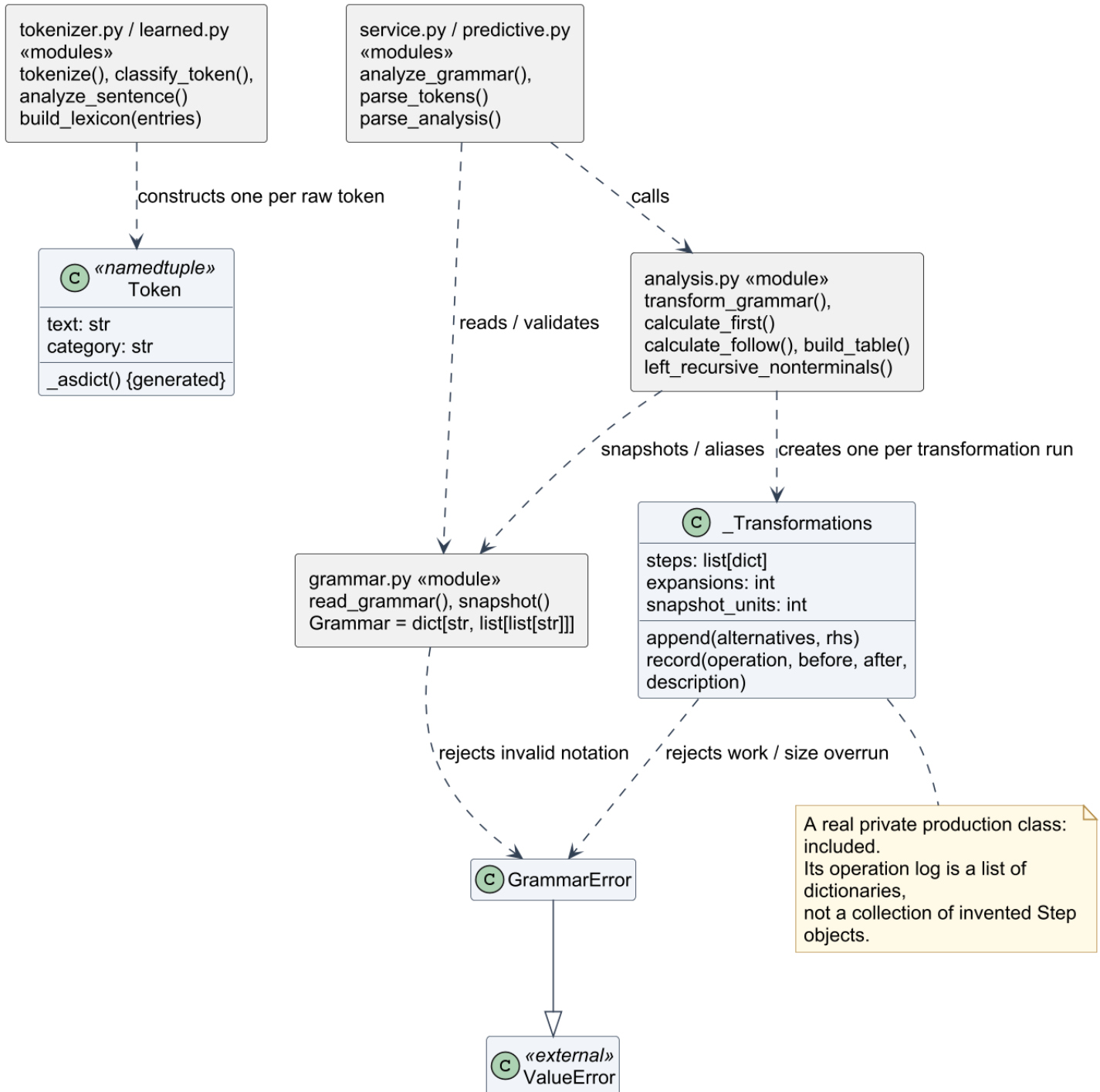
Authentication is mandatory for private APIs. CORS is not access control.
Local pre-hosting testing: browser -> loopback FastAPI, no public proxy required.
Vite is an optional development server, not the production static host.
Legacy desktop CSV/audio/profile are separate and never adopted by a hosted account.
Disk backup requires identity data too; workspace ZIPs alone cannot recover accounts.
Single durable host only: no ephemeral disk or implied multi-host database cluster.

Desktop application classes and audio ownership



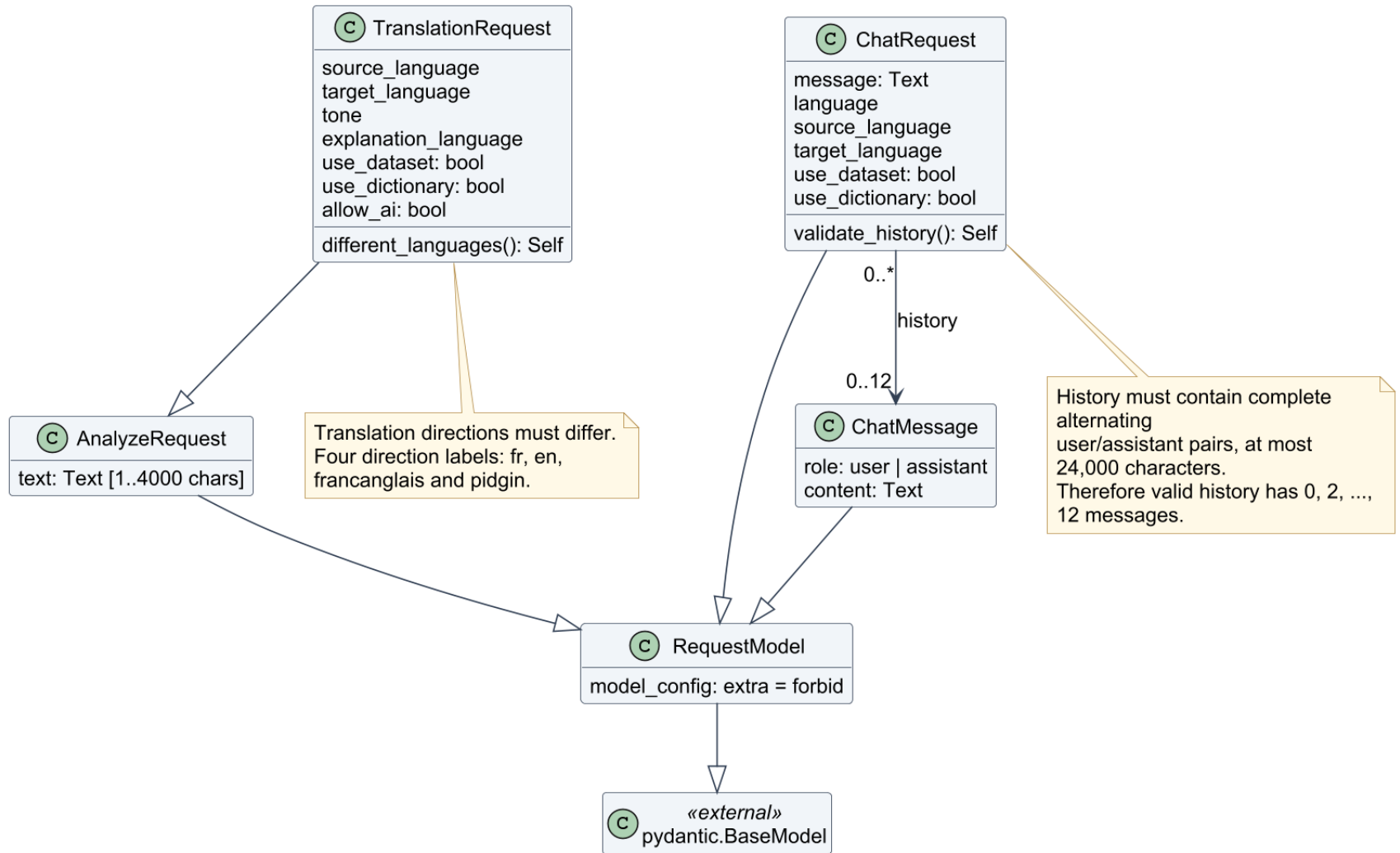
Five production classes; library types are explicitly external.
Per-instance target multiplicities express stored references.
Audio filenames are not object ownership or database foreign keys.

Compiler runtime classes and function modules



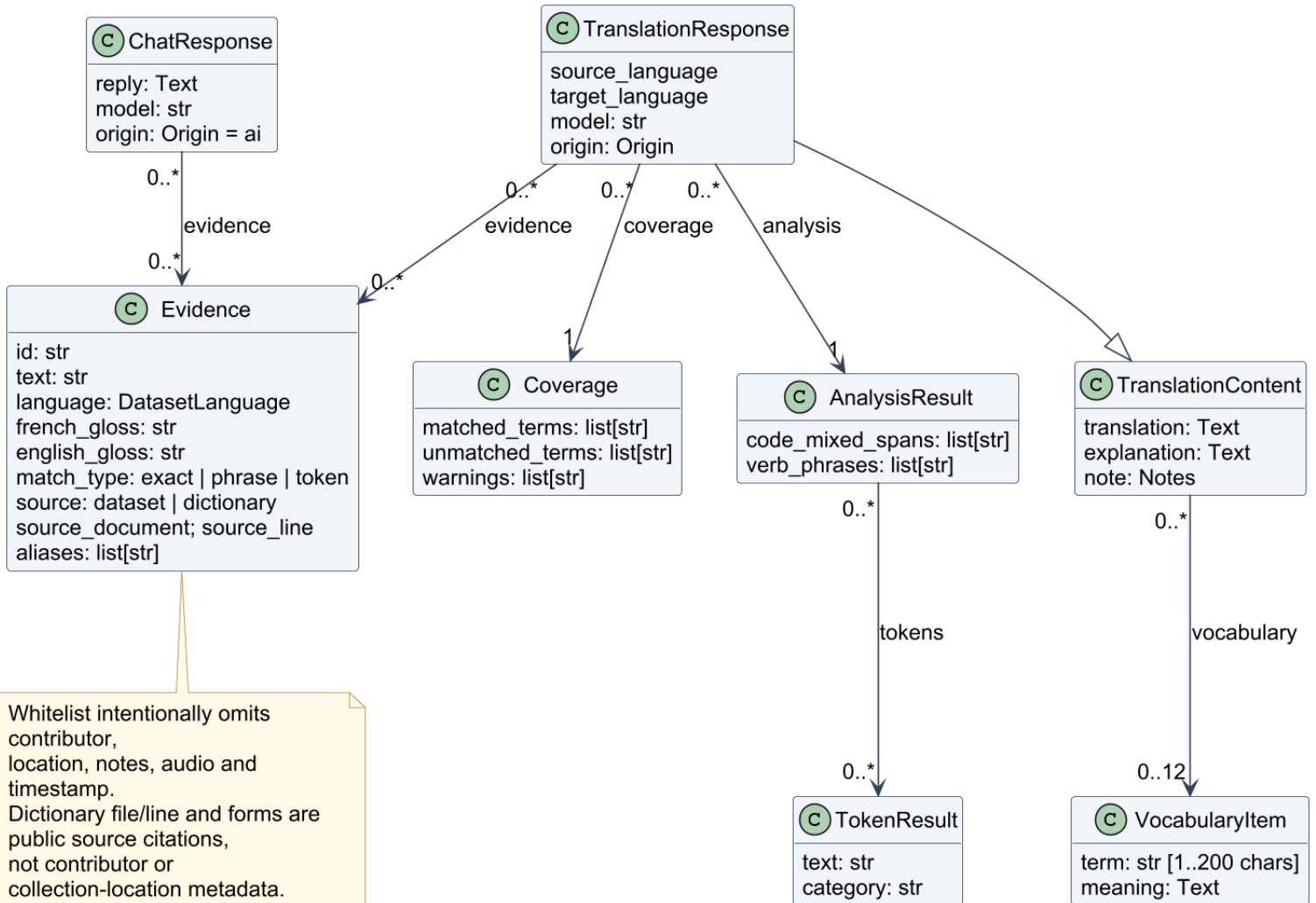
Token is an application-defined runtime class generated by namedtuple. Grammar, Production, analysis results and parse traces are aliases/dictionaries. There is no LexerService, Grammar class, Parser class or compiler database. Lifetimes of function-local values are not drawn as class compositions.

API request inheritance and conversation validation



All five primary classes live in backend.schemas.
Text is a constrained string alias, not an additional class.
Entry, import and coursework request subclasses are in detail views.

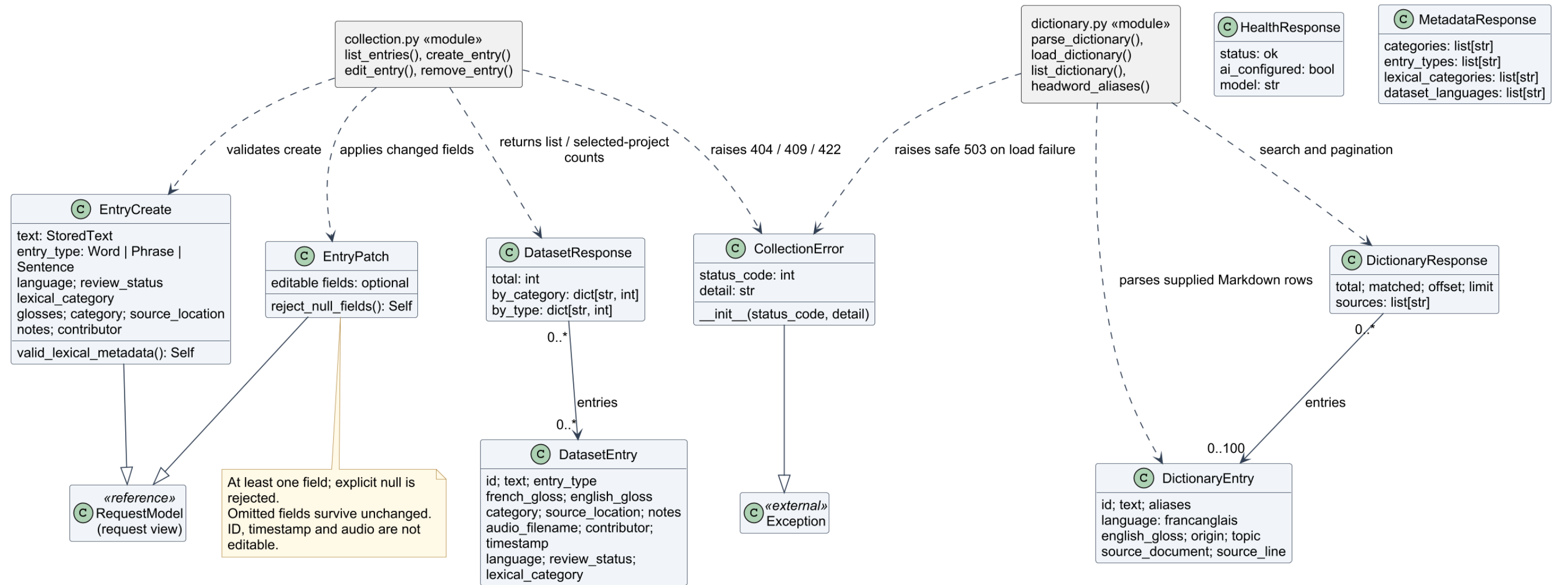
Translation and analysis result DTOs



Whitelist intentionally omits contributor, location, notes, audio and timestamp. Dictionary file/line and forms are public source citations, not contributor or collection-location metadata. Retrieval emits at most 8 evidence records, but these DTO list types alone do not enforce 8.

All eight primary classes live in backend.schemas. Each directly extends pydantic.BaseModel except TranslationResponse. Associations describe nested values, not exclusive lifetimes; Pydantic values may be shared/reused by Python callers. Validation/serialization use inherited model_validate/model_dump.

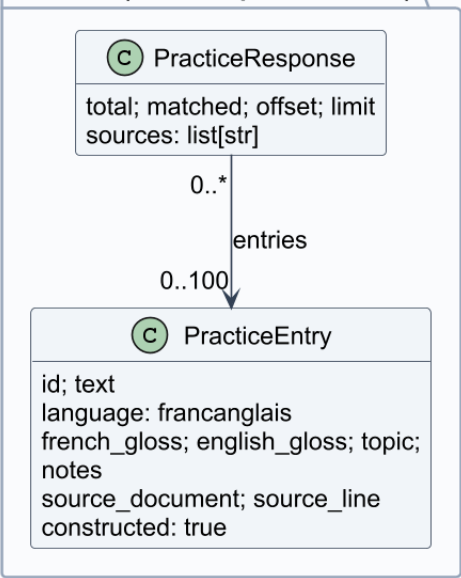
Separate collection and reference dictionary DTOs



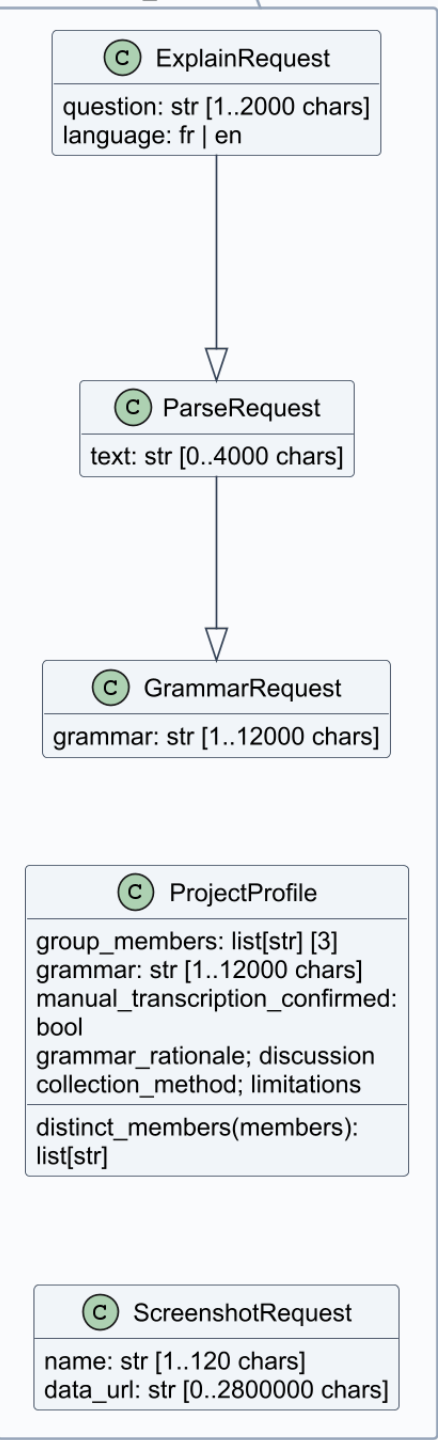
Import models and preserved academic request/result classes

backend

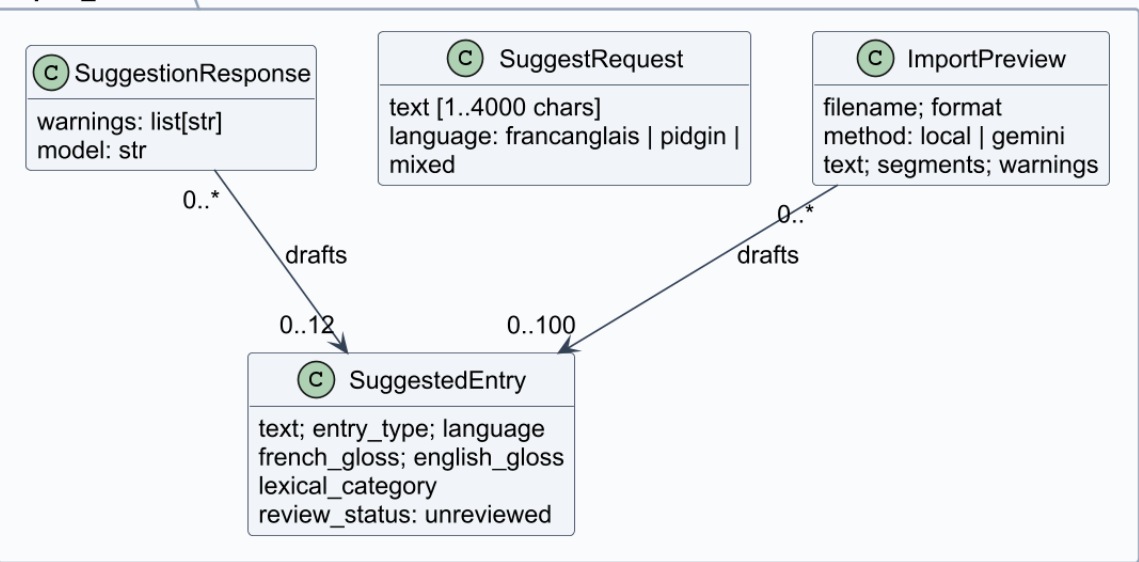
schemas (archived practice route)



coursework_models

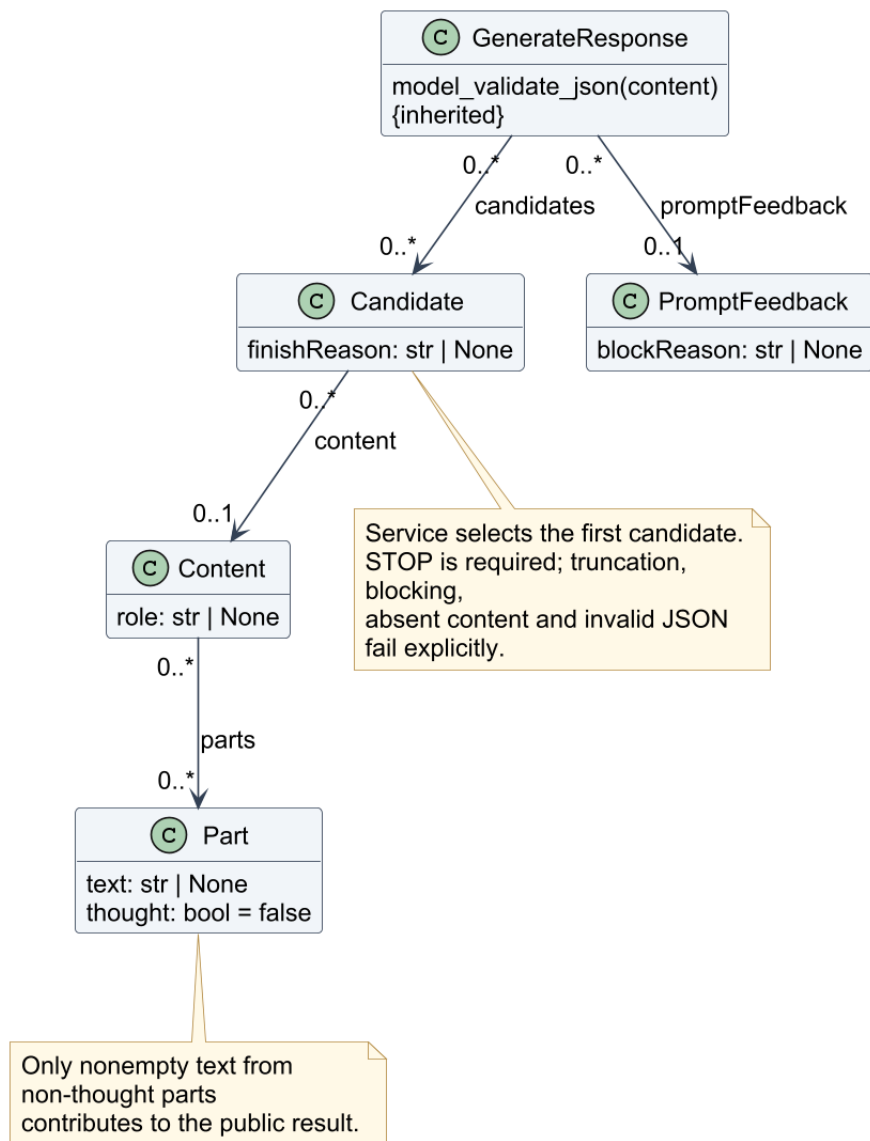


import_models



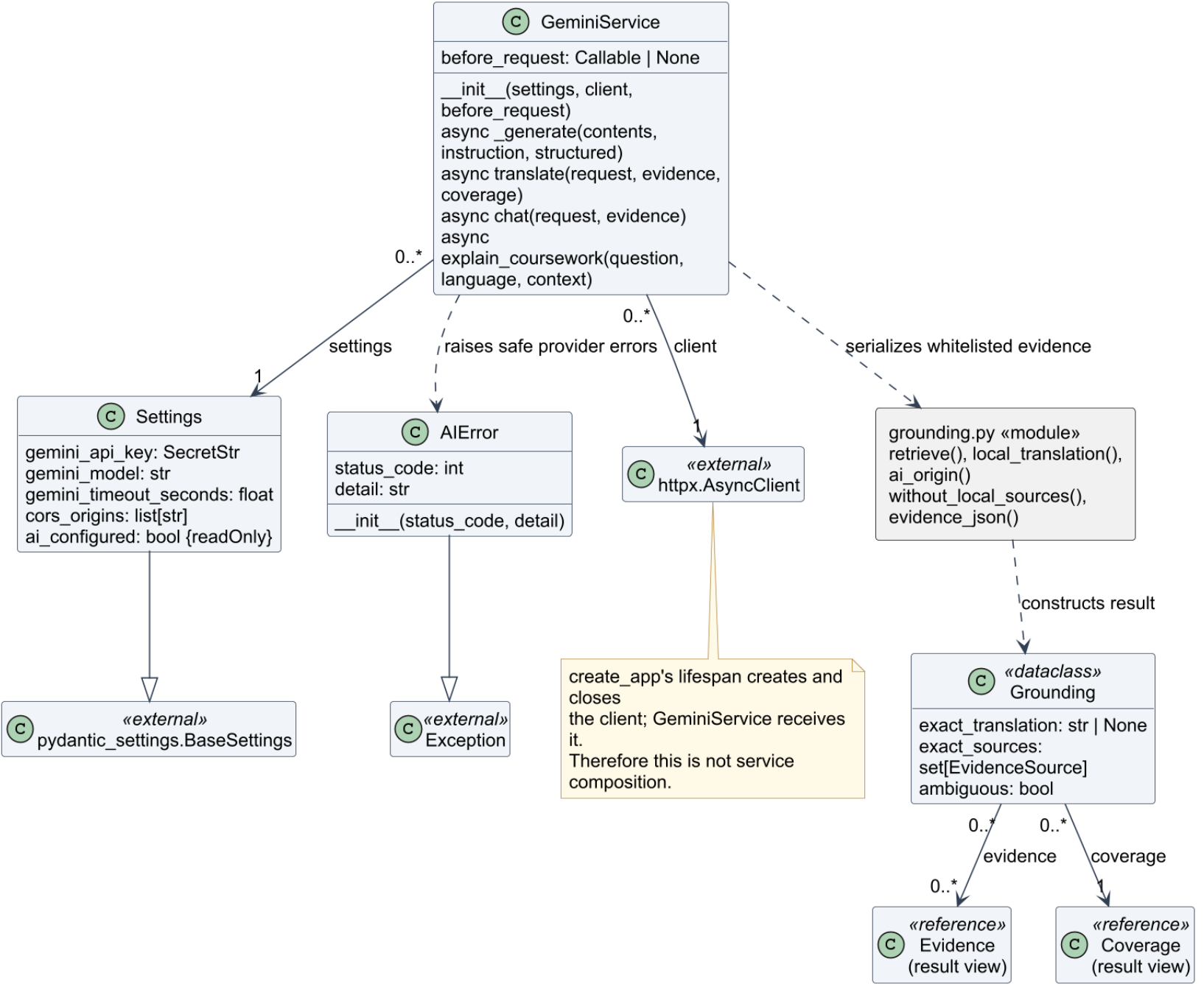
SuggestRequest and all coursework classes inherit RequestModel (directly, except ParseRequest and ExplainRequest as drawn). Other import classes directly inherit BaseModel. PracticeEntry and PracticeResponse also inherit BaseModel. SuggestedEntry additionally forbids extra fields; approval cannot be imported. Three member strings may initially be blank; nonblank names must be distinct. UI type aliases/interfaces are not additional runtime model classes. Academic DTOs remain in source but are not exposed by the hosted application.

Gemini wire-response validation classes



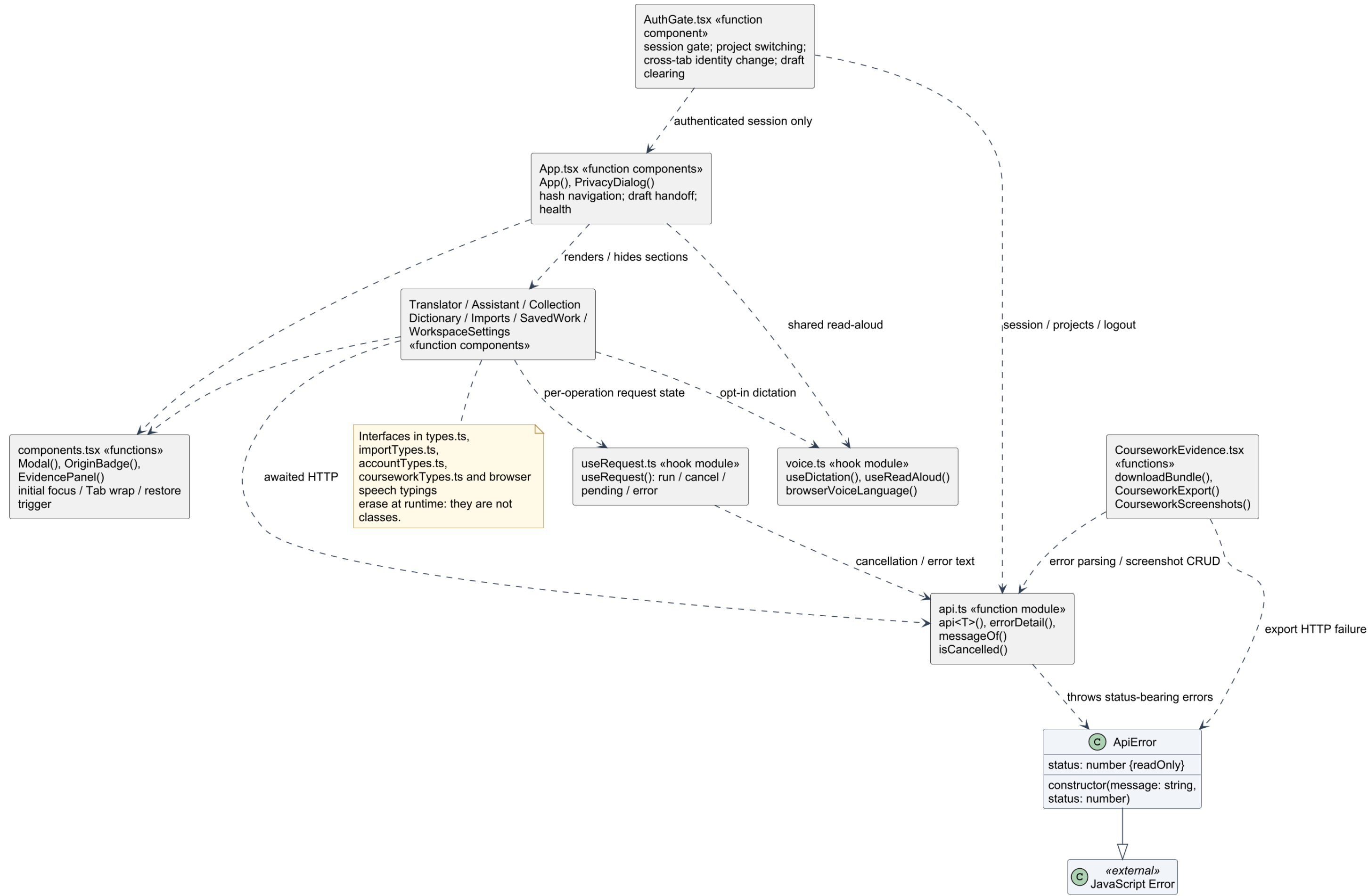
All five primary classes live in `backend.gemini` and extend `BaseModel`.
Optional fields/list defaults justify 0..1 and 0..*.
They validate upstream wire data, not stored corpus entities.
These classes are distinct from public `TranslationContent/ChatResponse` DTOs.

Actual provider service and local grounding result



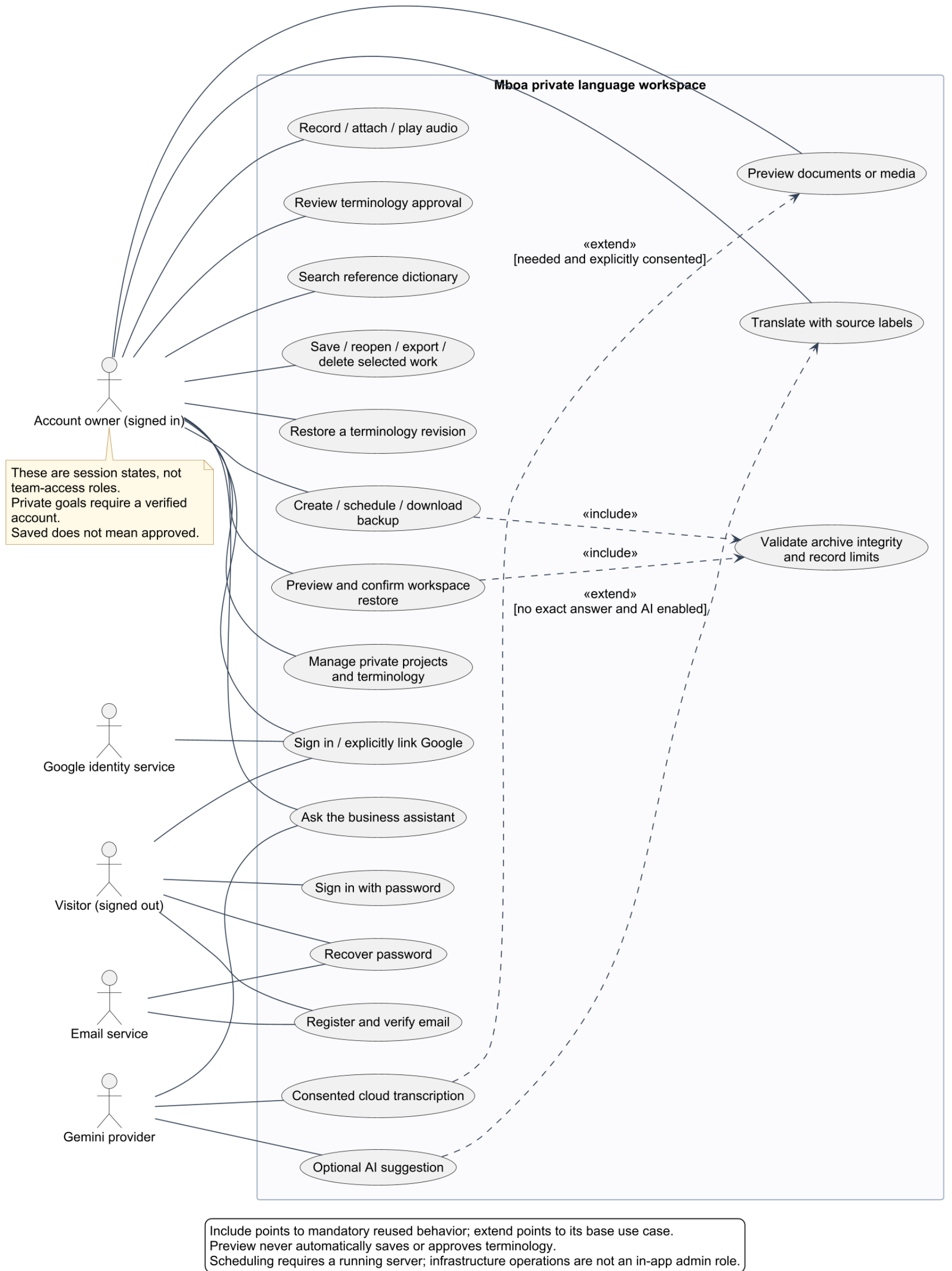
GeminiService is a real class. Retrieval and route handlers are functions. Grounding is a value holder, not a hidden training or translation service. At most 8 selected evidence records are emitted by retrieve(). Hosted quota accounting runs before every actual provider attempt, including retry.

Frontend class and function-component structure

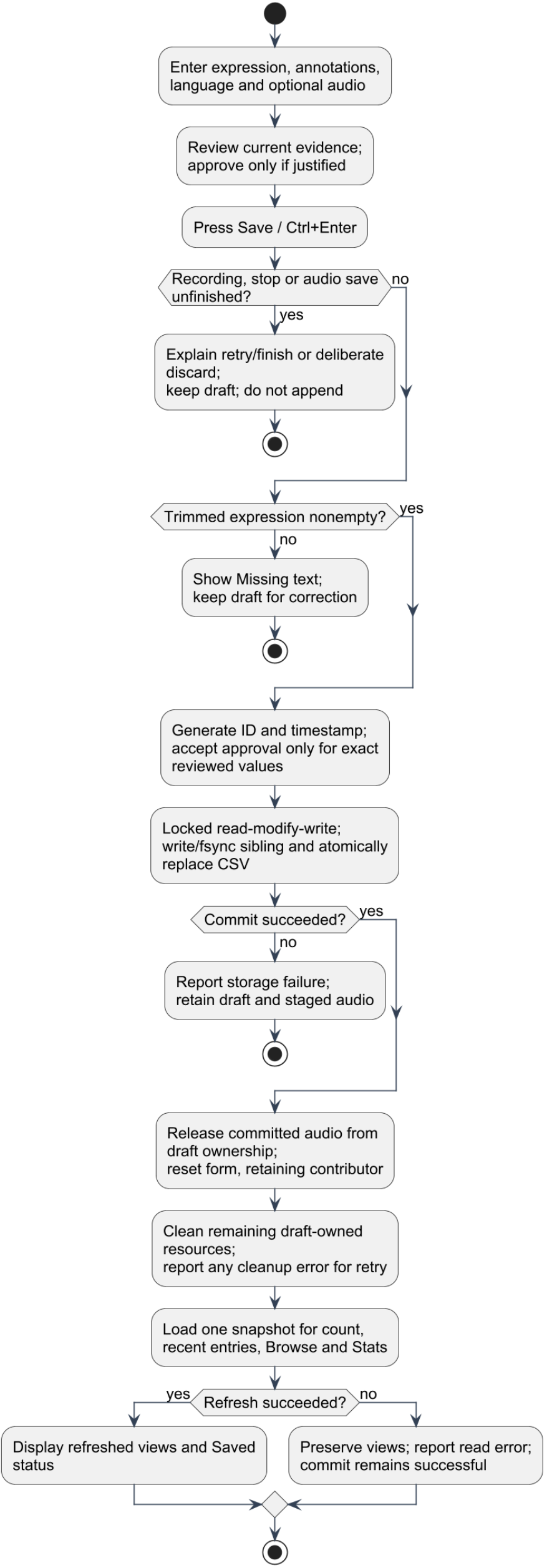


ApiError is the only authored production TypeScript class.
React function components and hooks must not be recast as service classes.
Module dependencies have no association multiplicities.
CourseworkEvidence is preserved archive code, not a business navigation destination.
api.ts attaches CSRF/project context and rejects responses from older account generations.

Hosted business user goals and supporting services

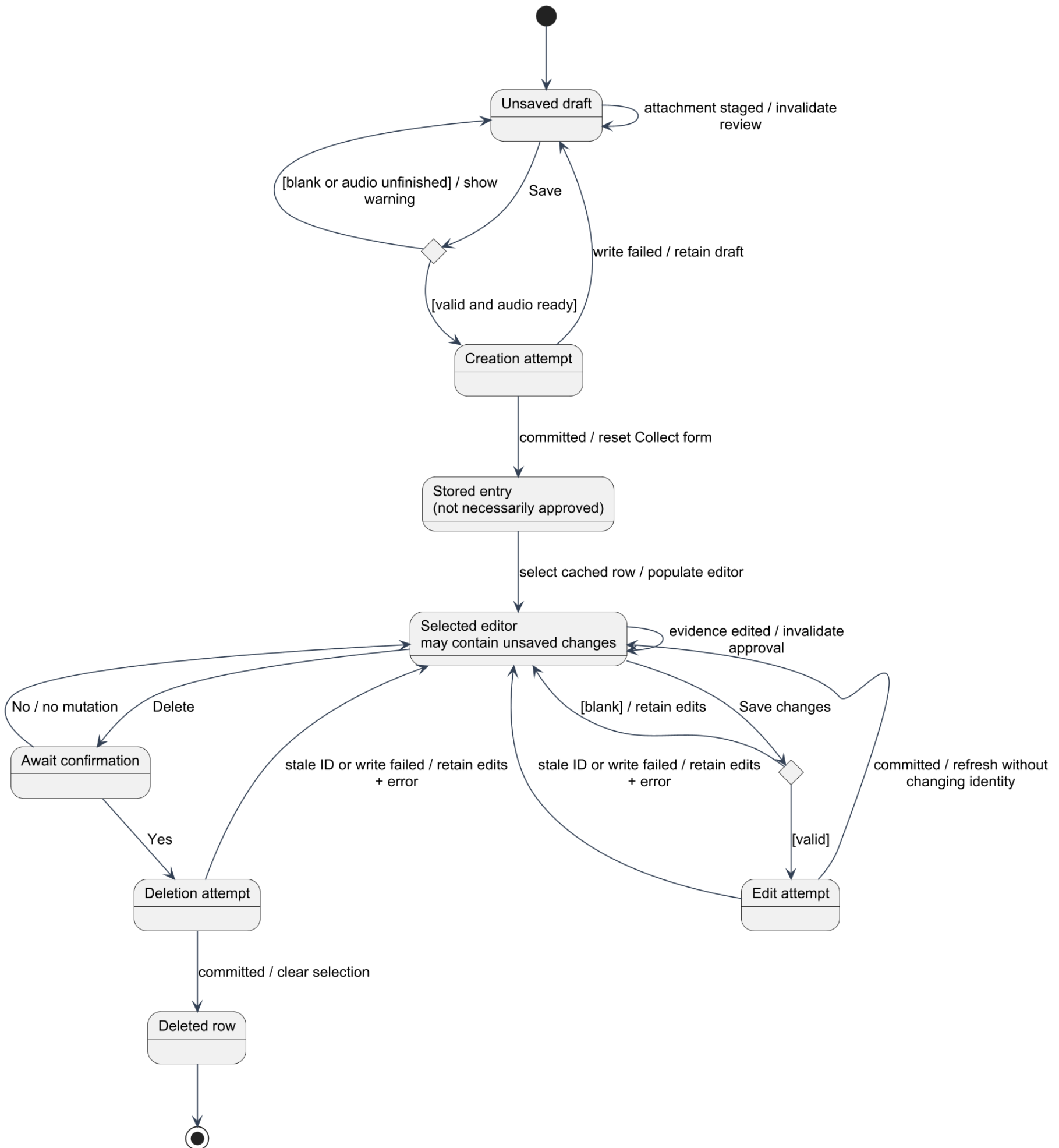


One desktop save attempt, including failure boundaries



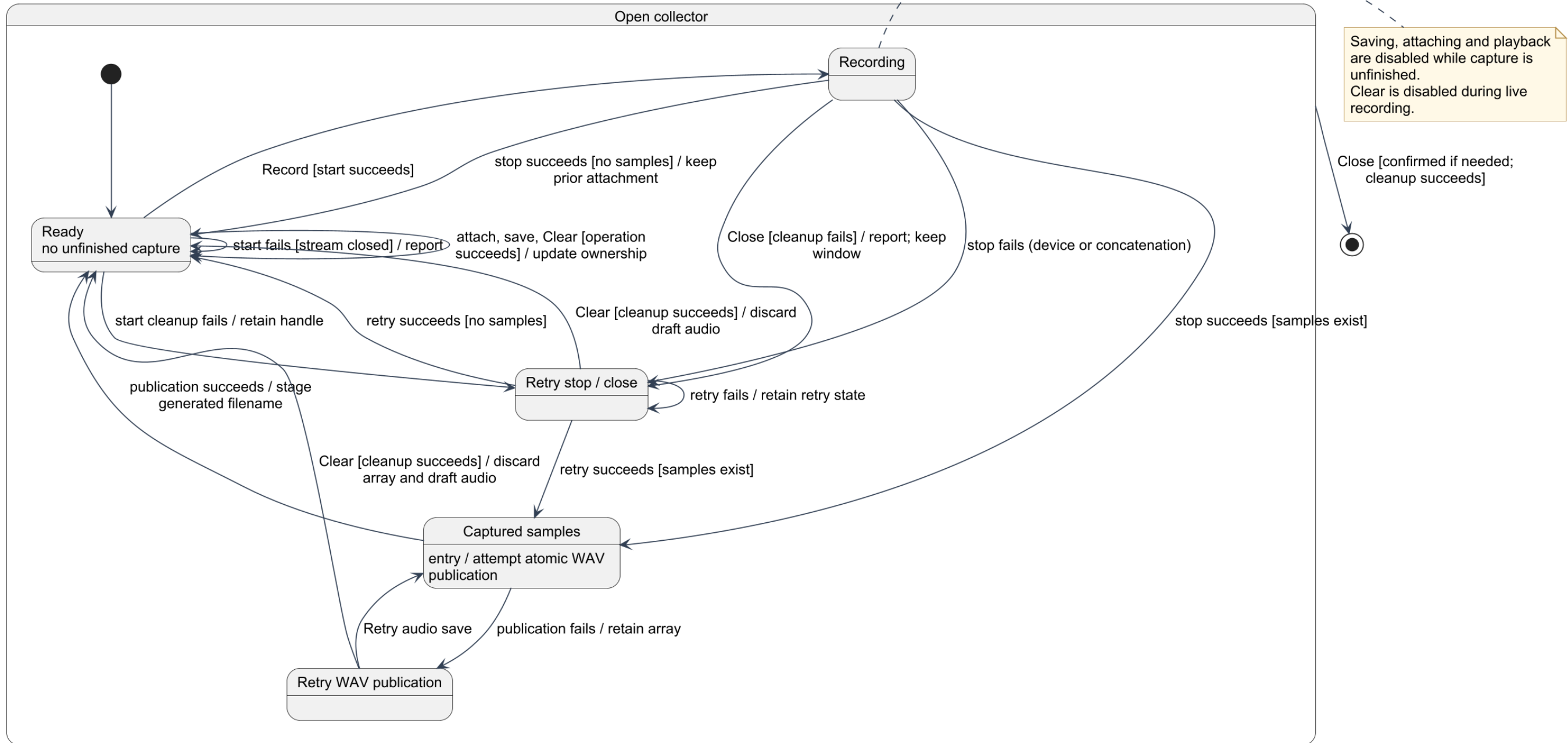
Diamonds represent exclusive guarded decisions, not parallel work.
One final node ends this attempt, not the collector application.
Duplicate warnings are advisory on desktop; API duplicates return 409.
File staging is separate; recording details appear in the sequence view.

Logical entry and selected-editor lifecycle



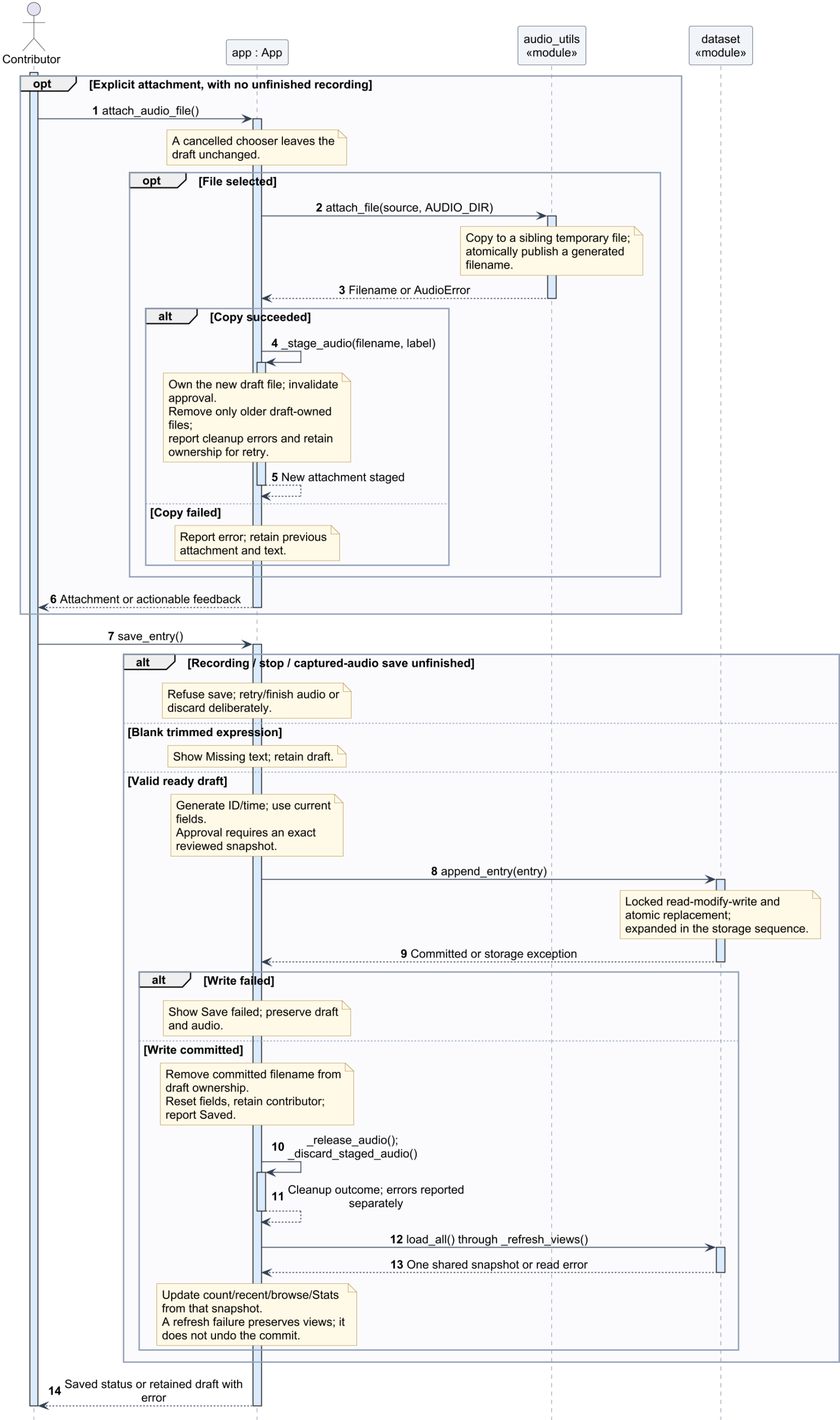
This is a logical row/editor model, not an implemented enum or whole-app lifecycle. The selected editor may outlive a row removed by another writer; stale errors are explicit. A successful commit is not reversed by later display/cleanup failure. Audio staging has a separate lifetime; deleting a saved row retains its recording.

Desktop capture readiness and retry states

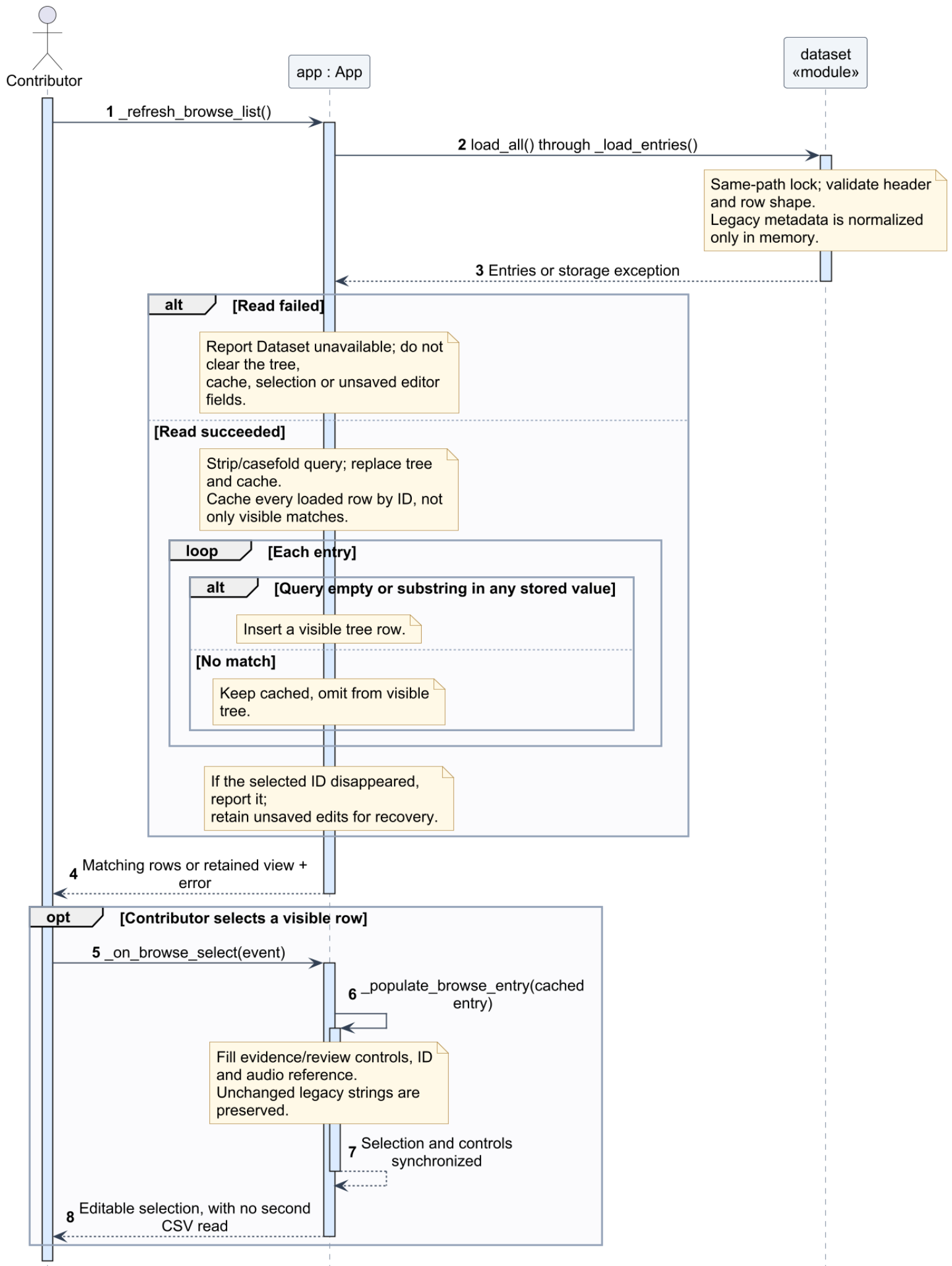


Conceptual states derive from recording, stream, pending-stop and captured-array fields. Ready may contain a staged filename; it does not mean an empty form. Failed cleanup keeps the window/retry state available, not a false closed status. Draft-owned files may be discarded; committed recordings are never owned by the draft.

Desktop attachment and truthful entry saving

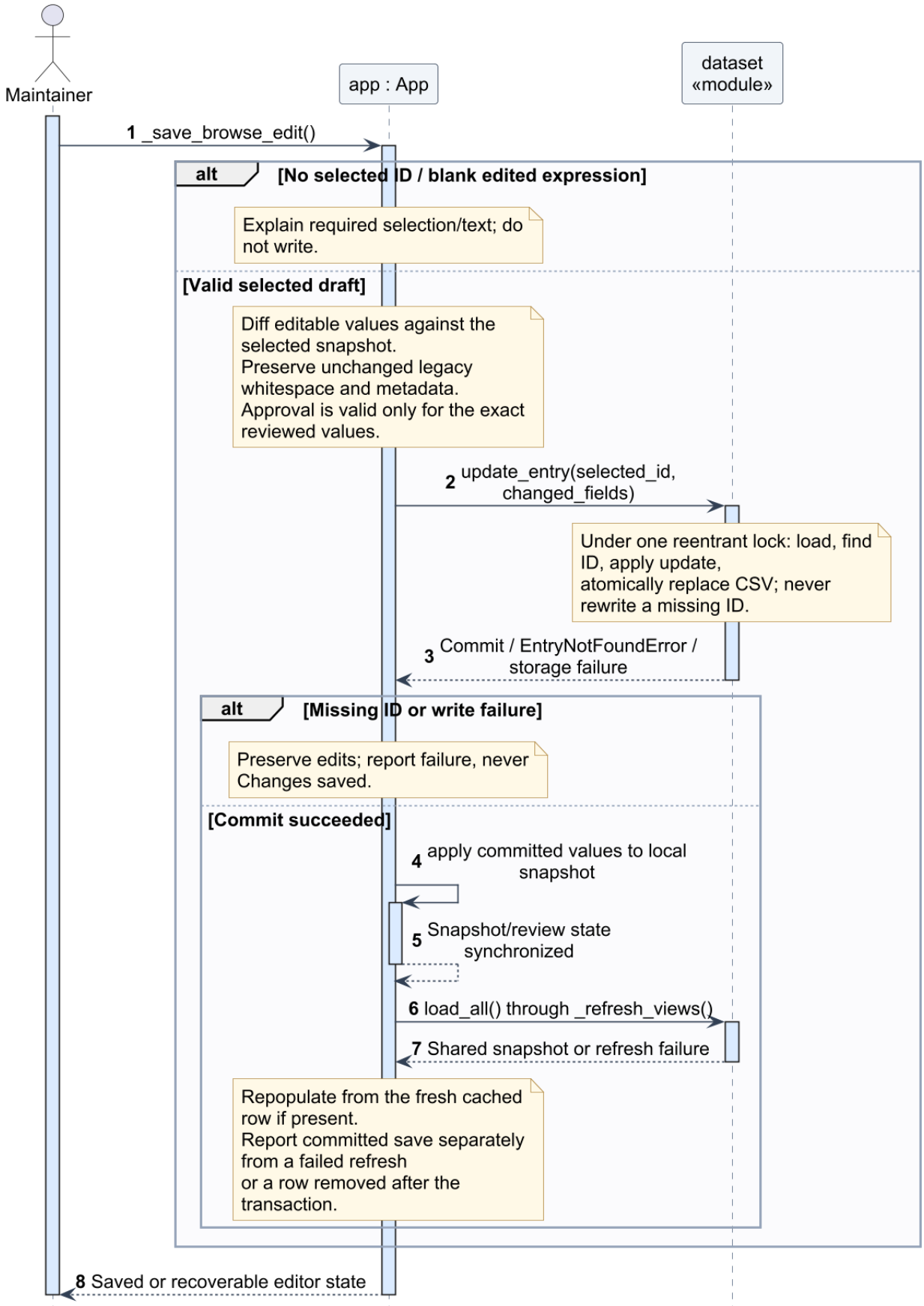


Desktop search, preserved views and cached selection



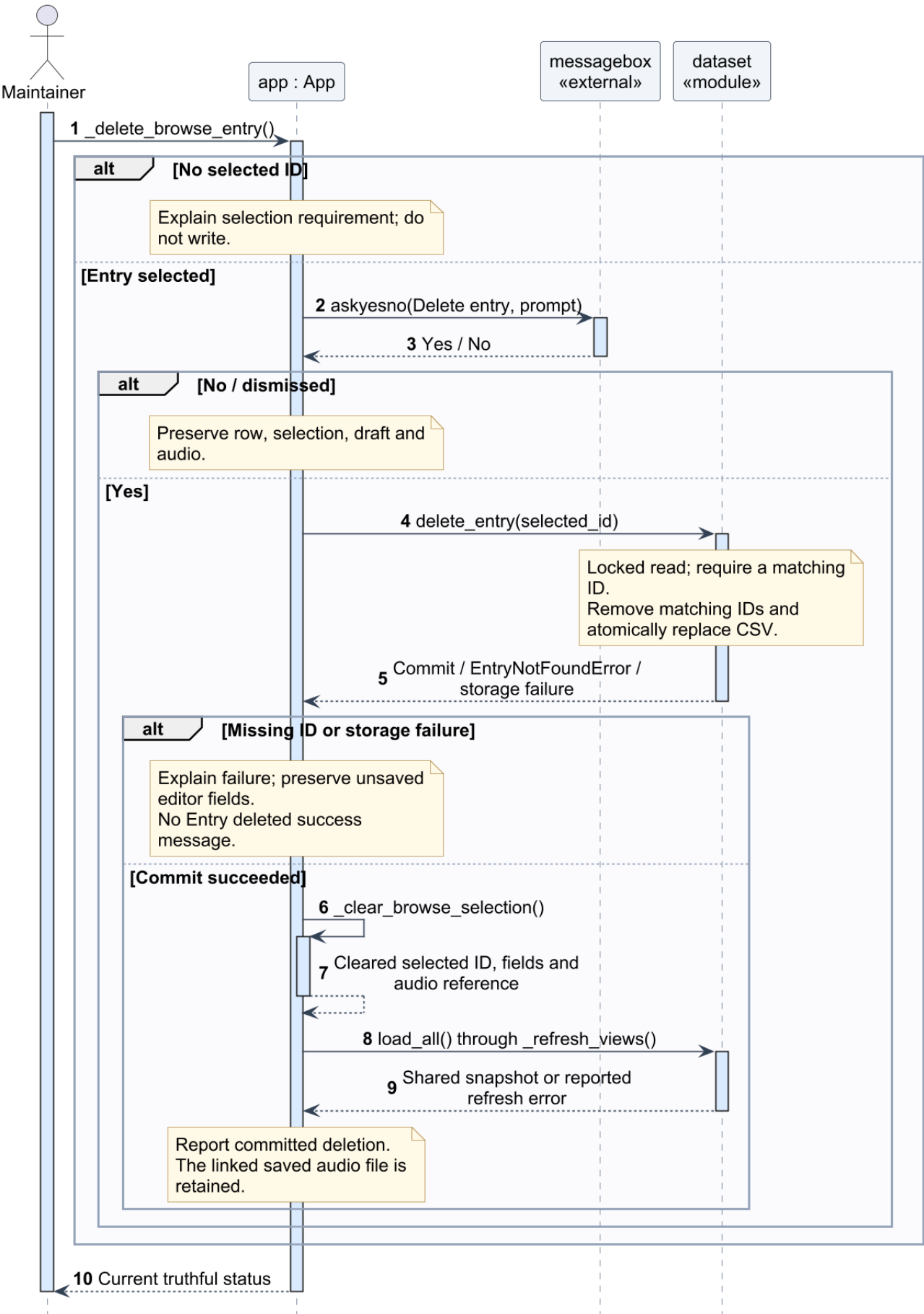
IDs should remain unique; manually corrupted identifiers are not repaired here. Filtering is read-only. Empty results are distinct from a failed read.

Desktop changed-field editing and review integrity



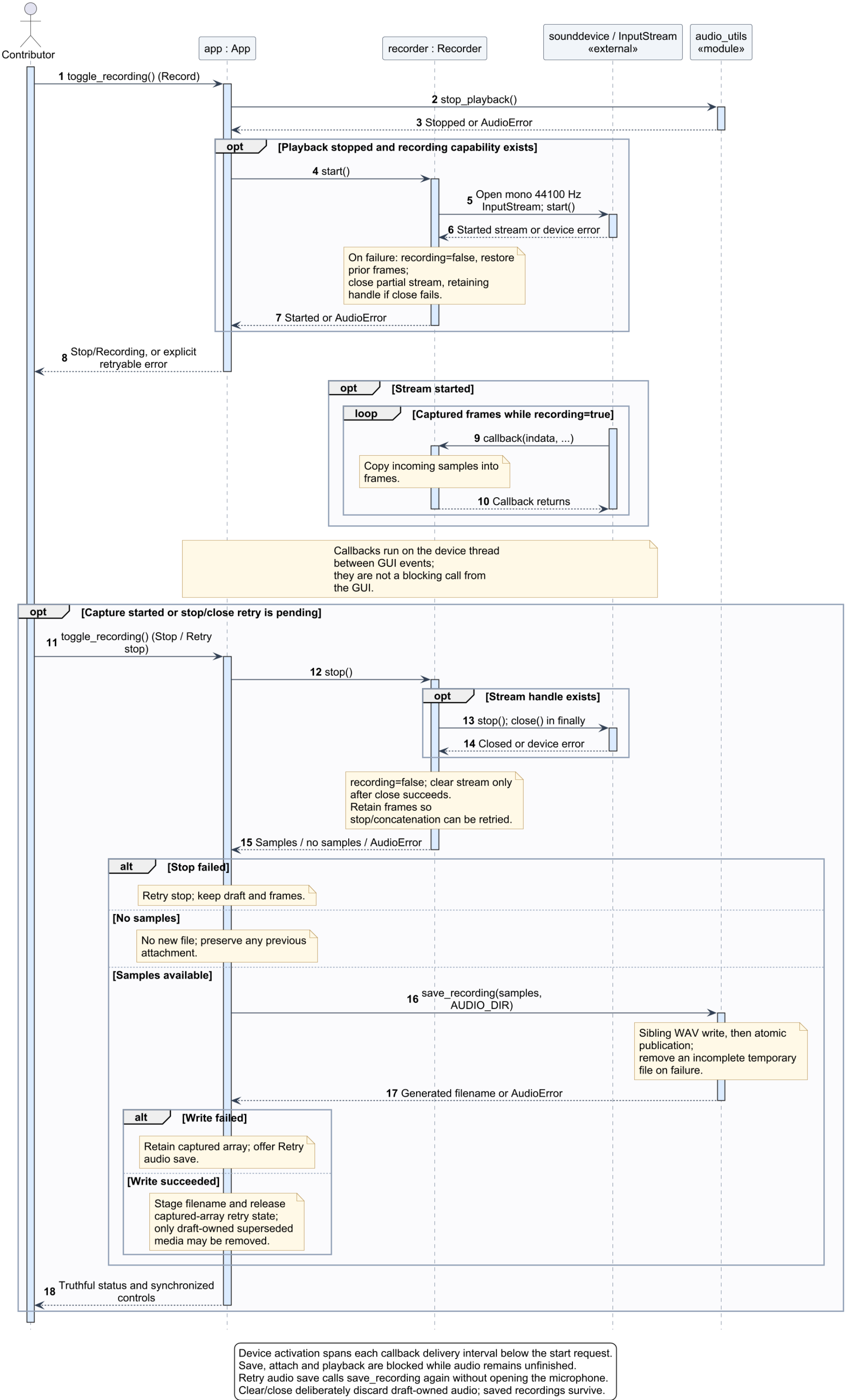
Desktop edits preserve ID, timestamp, contributor and audio reference. Text, glosses, topic, location, notes, language and lexical metadata are editable. Locks prevent interleaved writes, not semantic last-writer-wins conflicts.

Confirmed deletion without false success or media loss

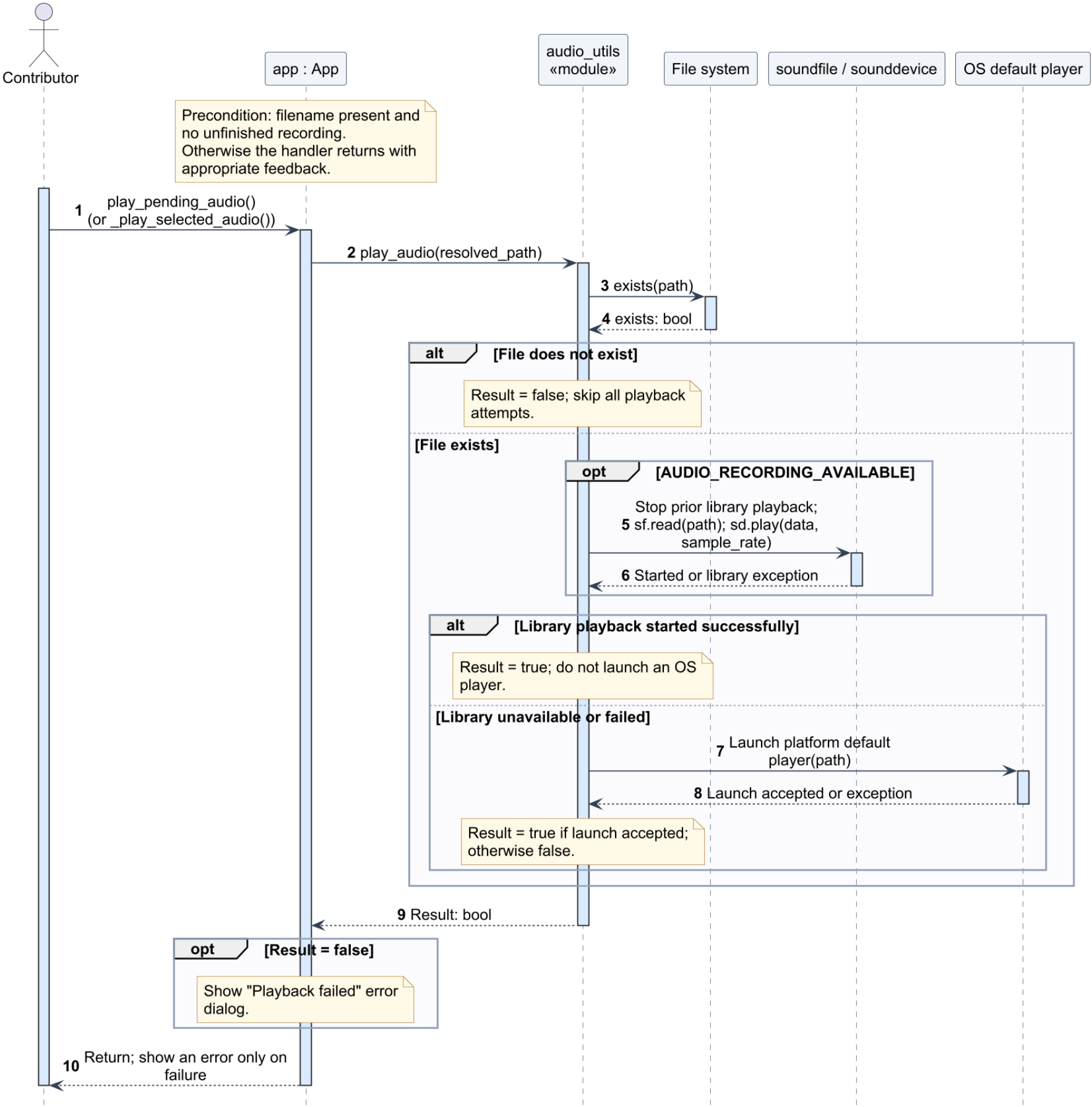


Confirmation is a decision, not an authorization or authentication check.
Atomic replacement protects write integrity; there is no undo stack.

Retryable microphone capture and WAV staging

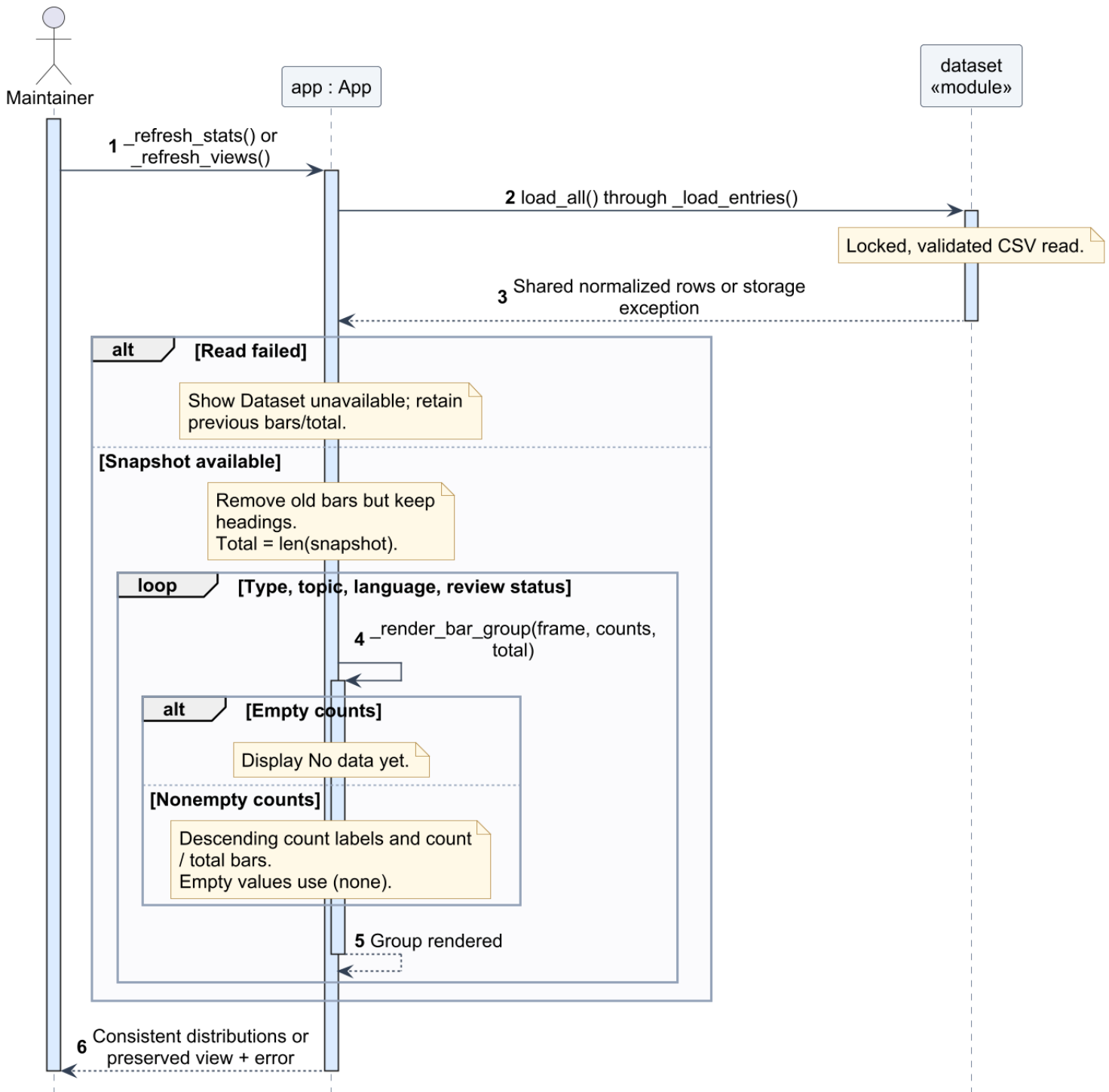


Play Audio with Library and OS Fallback



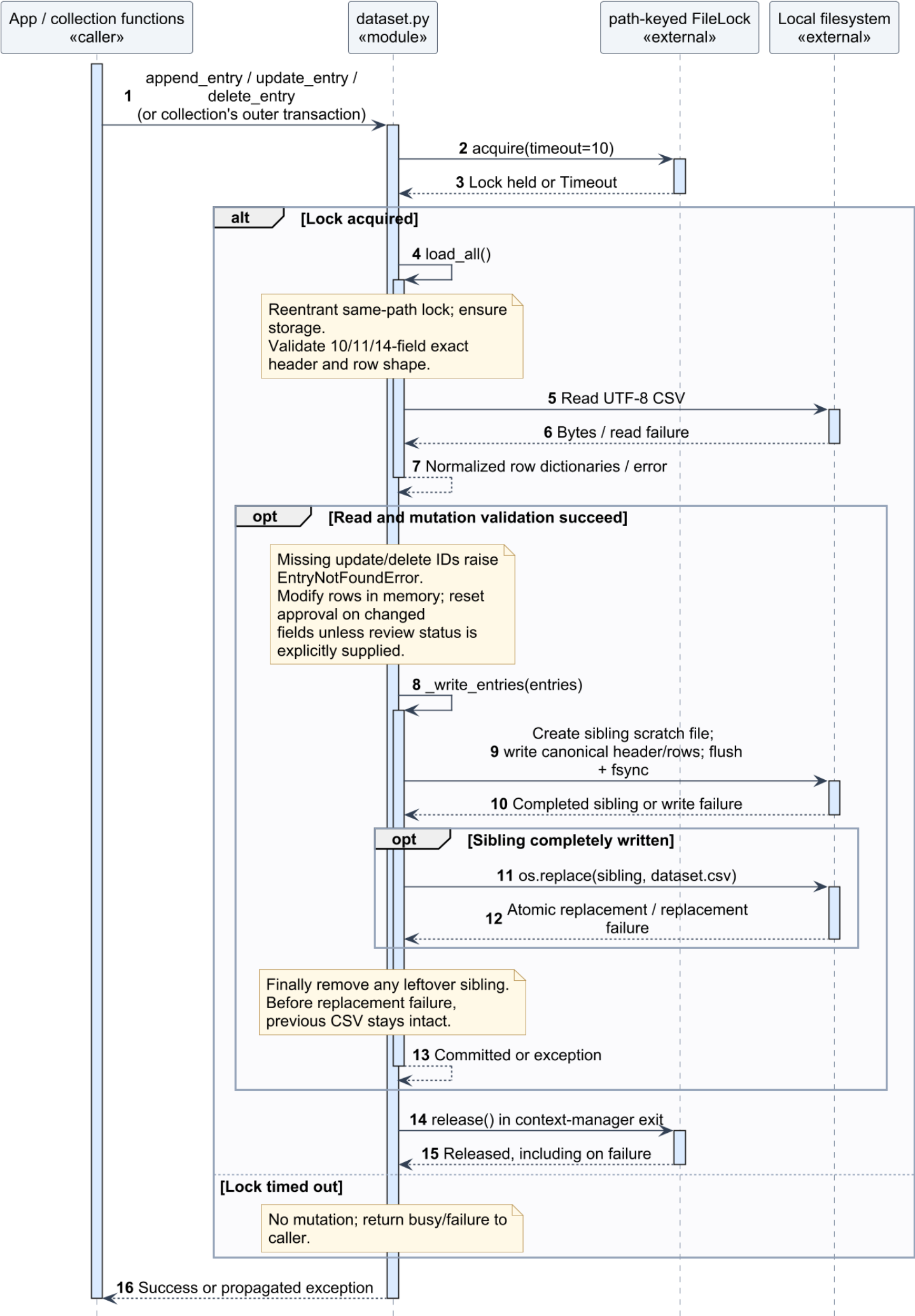
true means a playback request was accepted, not that sound was heard to completion. OS launch acceptance is not playback completion; media decoding is external. Expected audio errors are reported by the GUI; CSV is unchanged. Clear/close can stop in-process playback, not an external OS player's lifetime.

Consistent four-way statistics from one snapshot



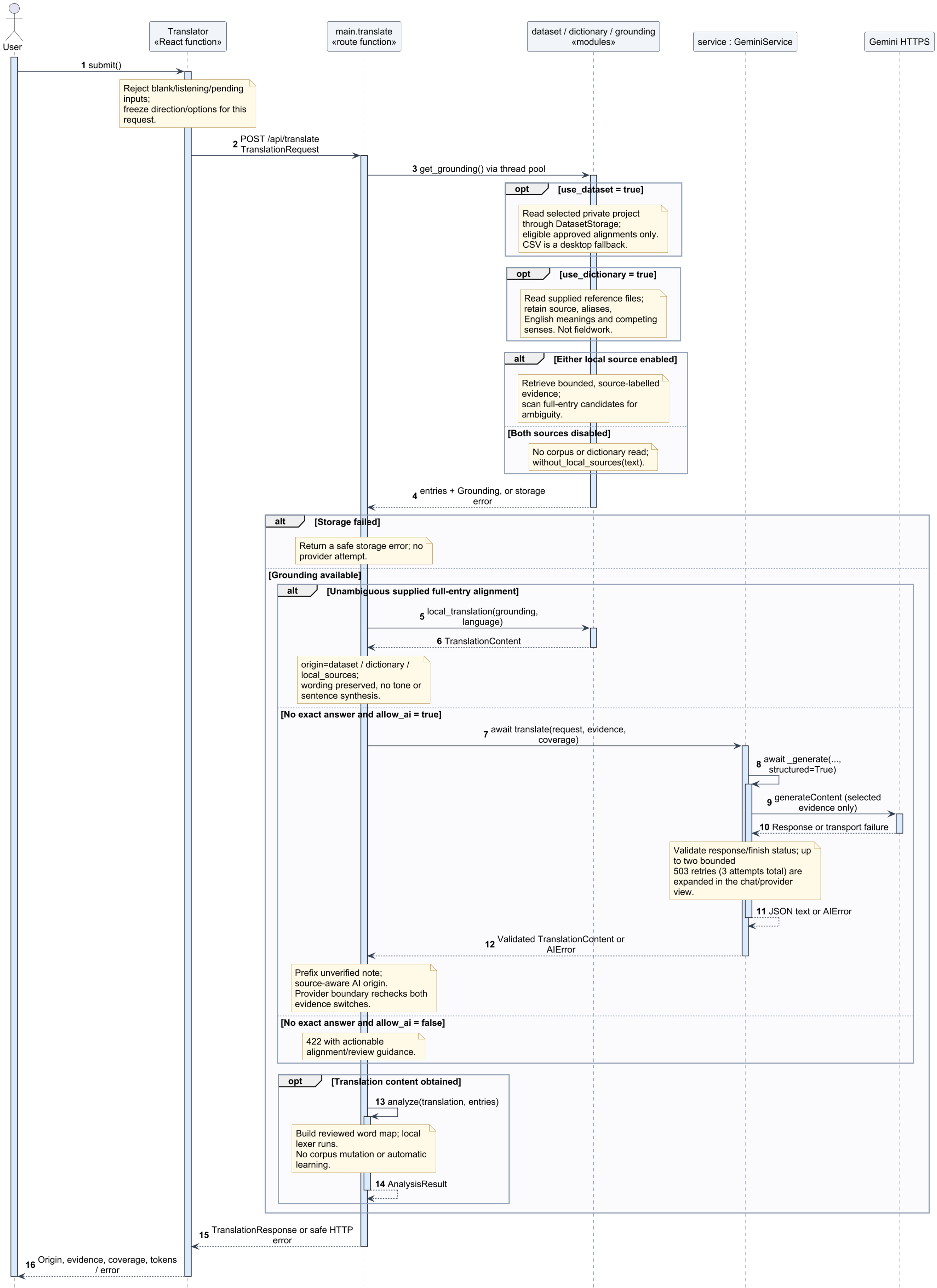
`_refresh_views` also supplies the same rows to count, recent and Browse views. Direct Stats refresh loads once; `count_by()` is not called by this handler. Performance evidence includes active-tab Tk idle rendering, not only CSV timing.

Shared locked CSV mutation and failure-safe replacement

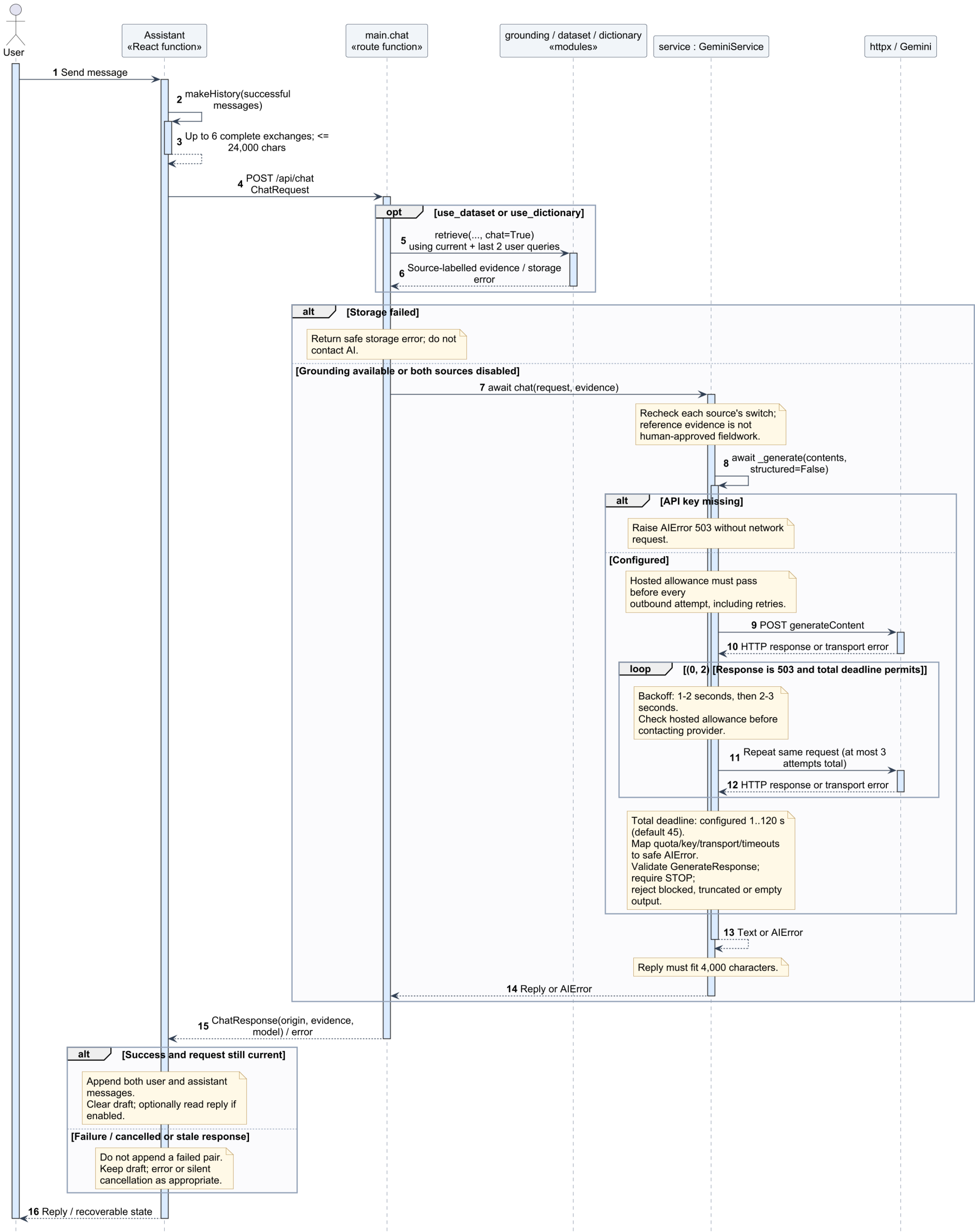


collection functions take the same outer lock around duplicate/ID checks and write. Lock activation shows the acquire/release call, not the duration of file ownership. Append also rewrites the full dataset; there is no unsafe direct append path. Protection covers cooperative users of this path, not external manual writers.

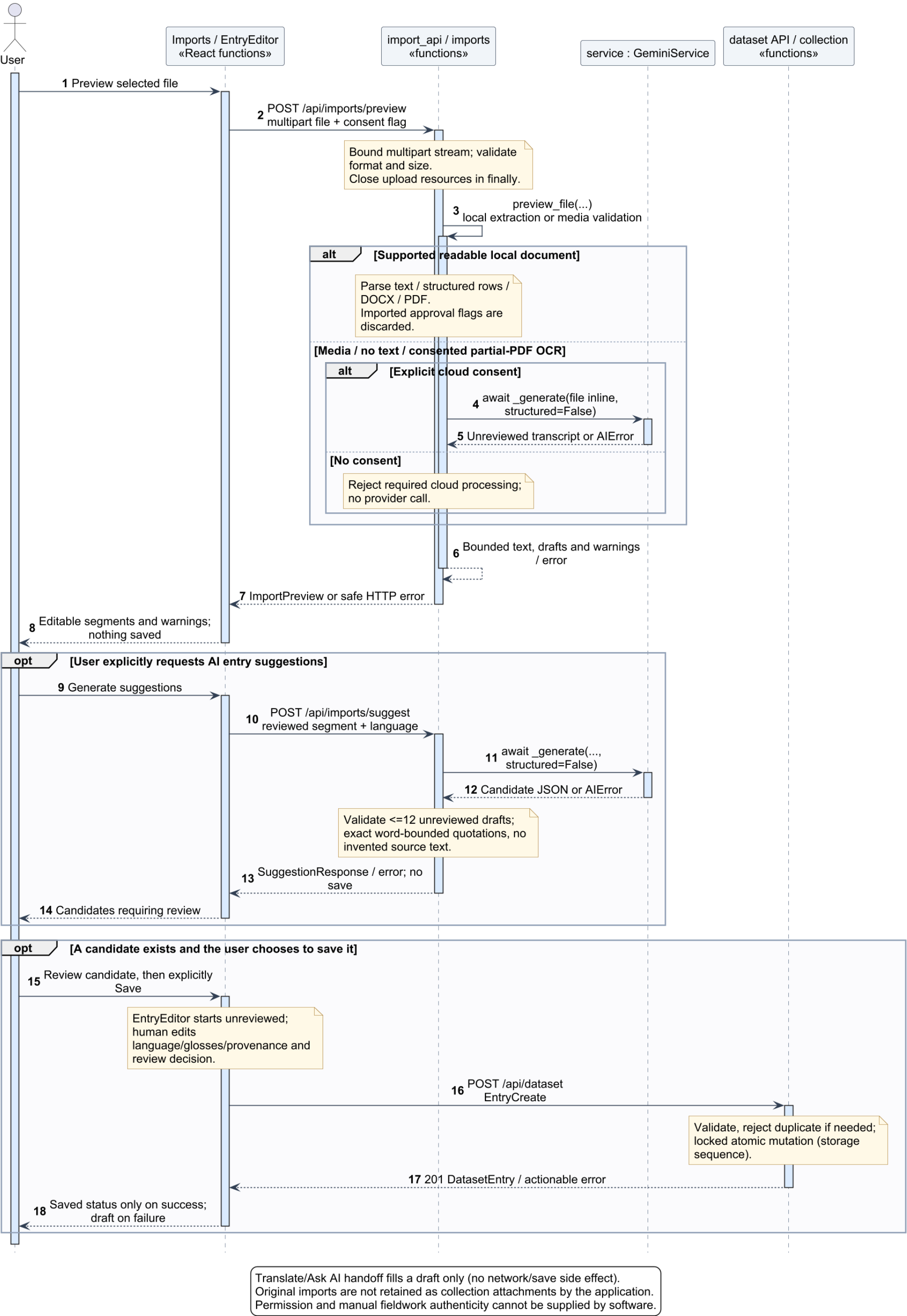
Web translation: enabled local sources, then optional AI



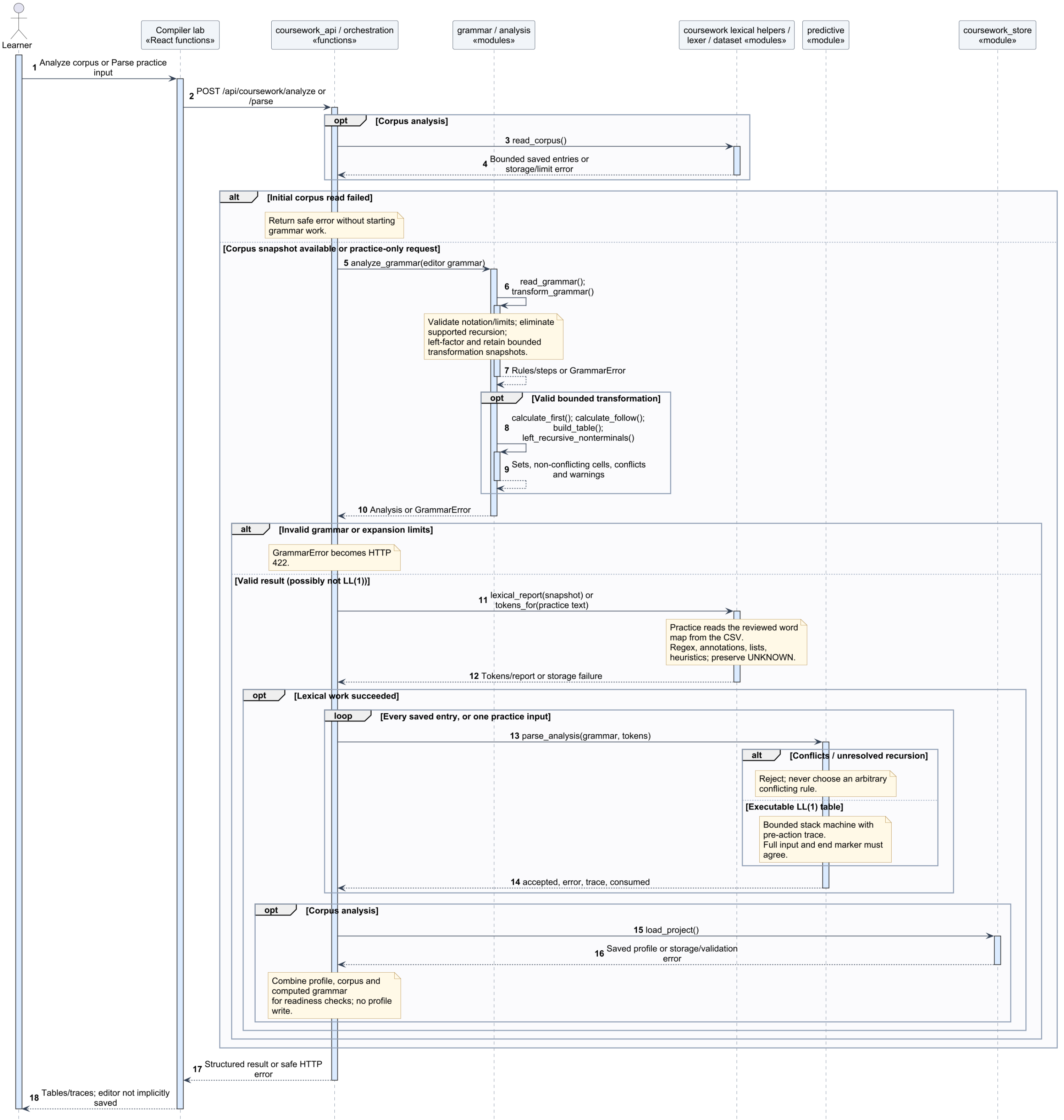
Explicit chat and bounded provider response handling



Import preview, explicit suggestions and human review/save

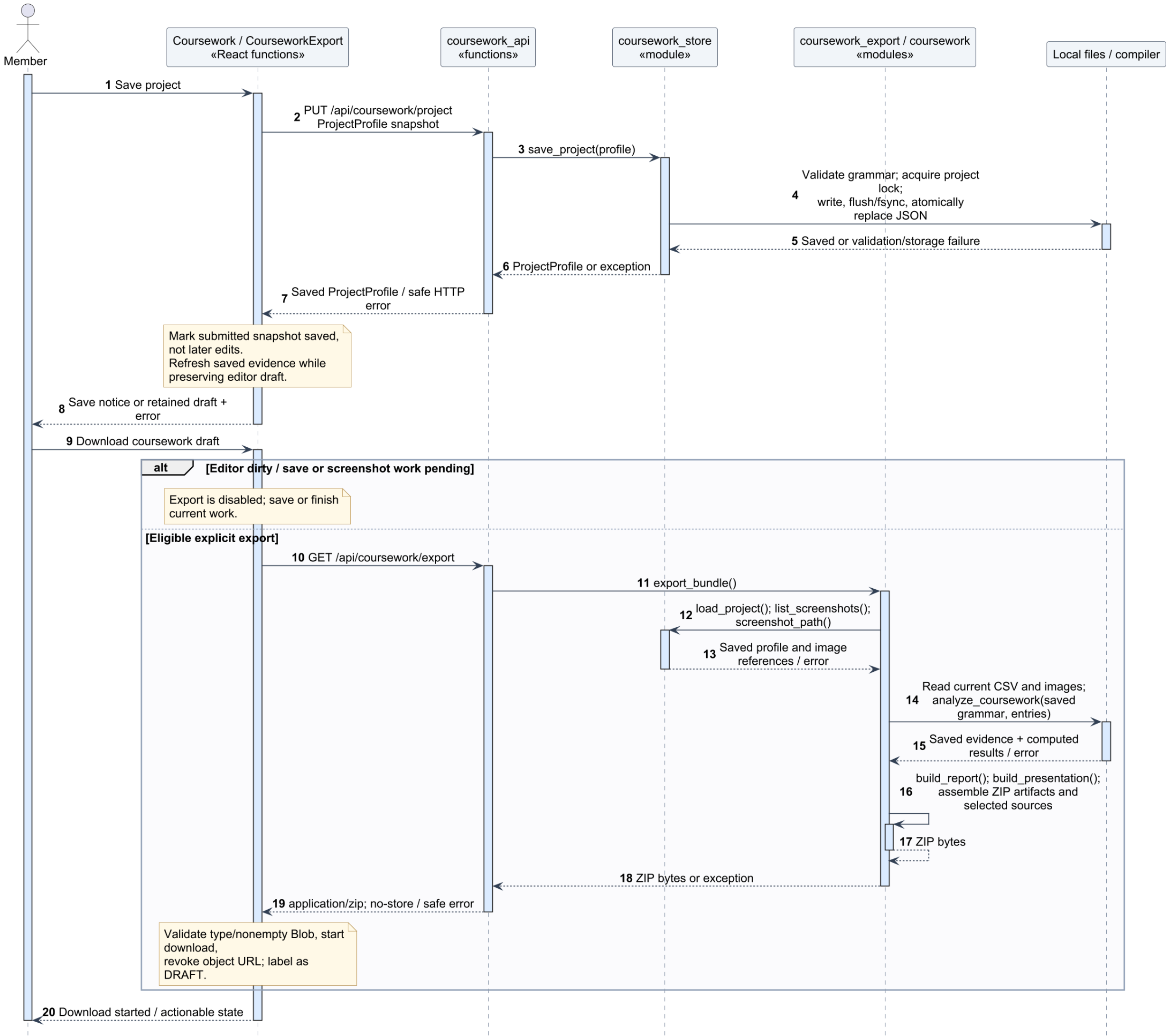


Deterministic grammar analysis and category-token parsing



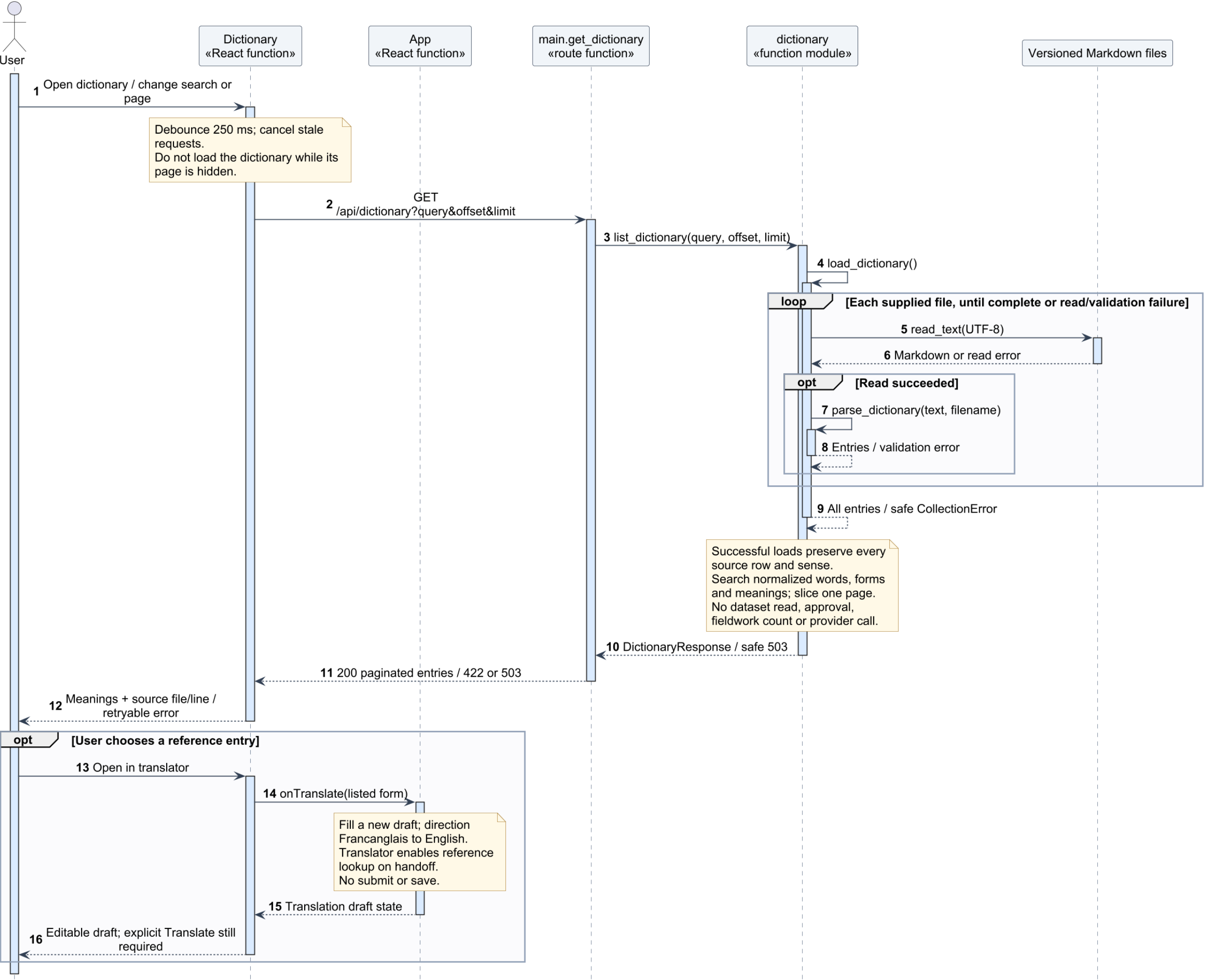
Module self-calls condense internal function boundaries.
No provider determines parsing. Corpus/grammar fit is not linguistic authenticity.
Existing corpus/media stay unchanged; absent storage can bootstrap an empty header.

Save coursework, then export only saved evidence

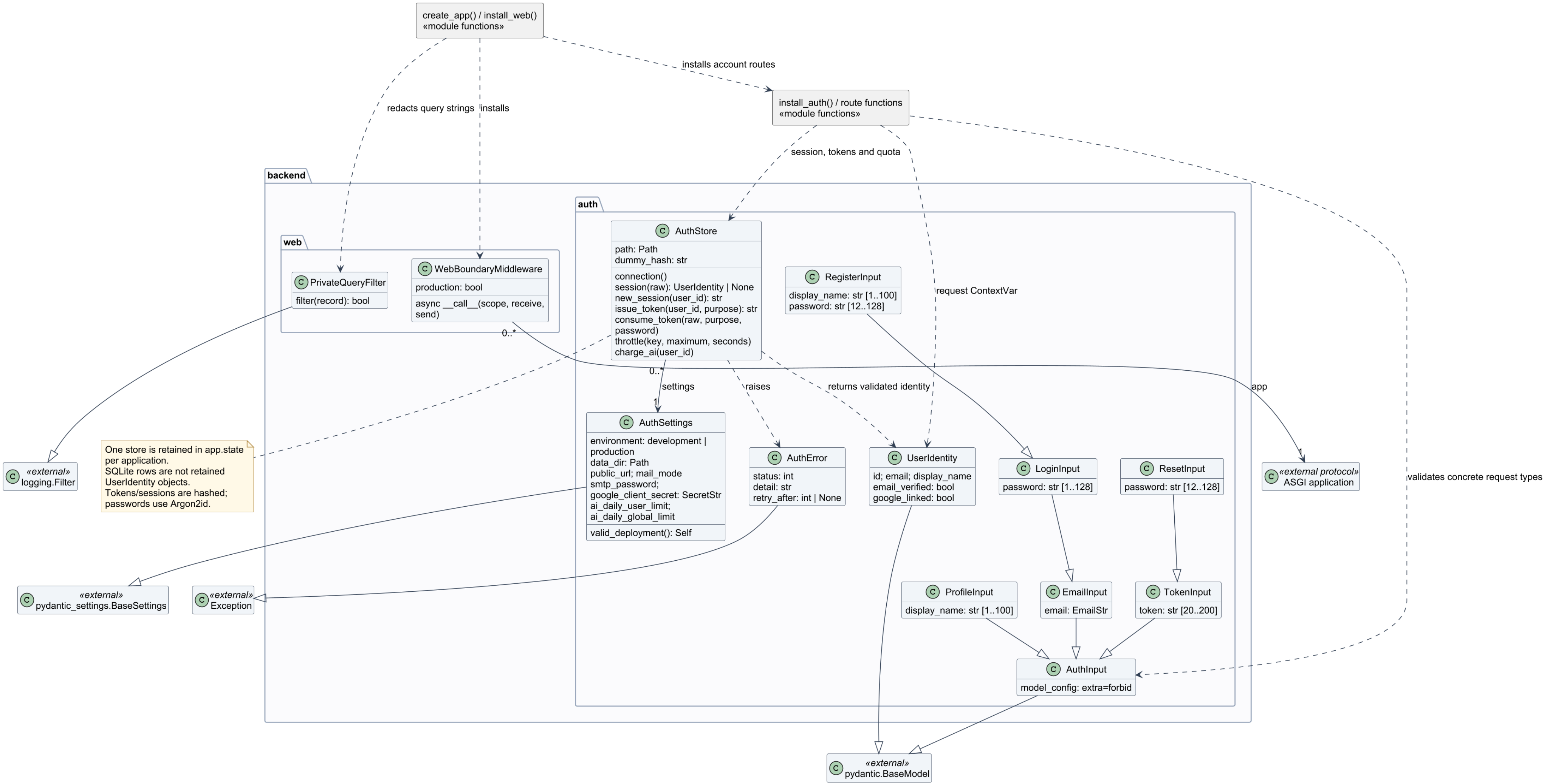


No unsaved practice input or AI answer is included.
ZIP includes actual corpus/provenance/profile: review before sharing.
Individual files are protected, but export is not a global multi-file transaction.
Missing research remains visible; 25 sections do not guarantee 25-30 final pages.

Reference lookup: separate sources, explicit draft handoff

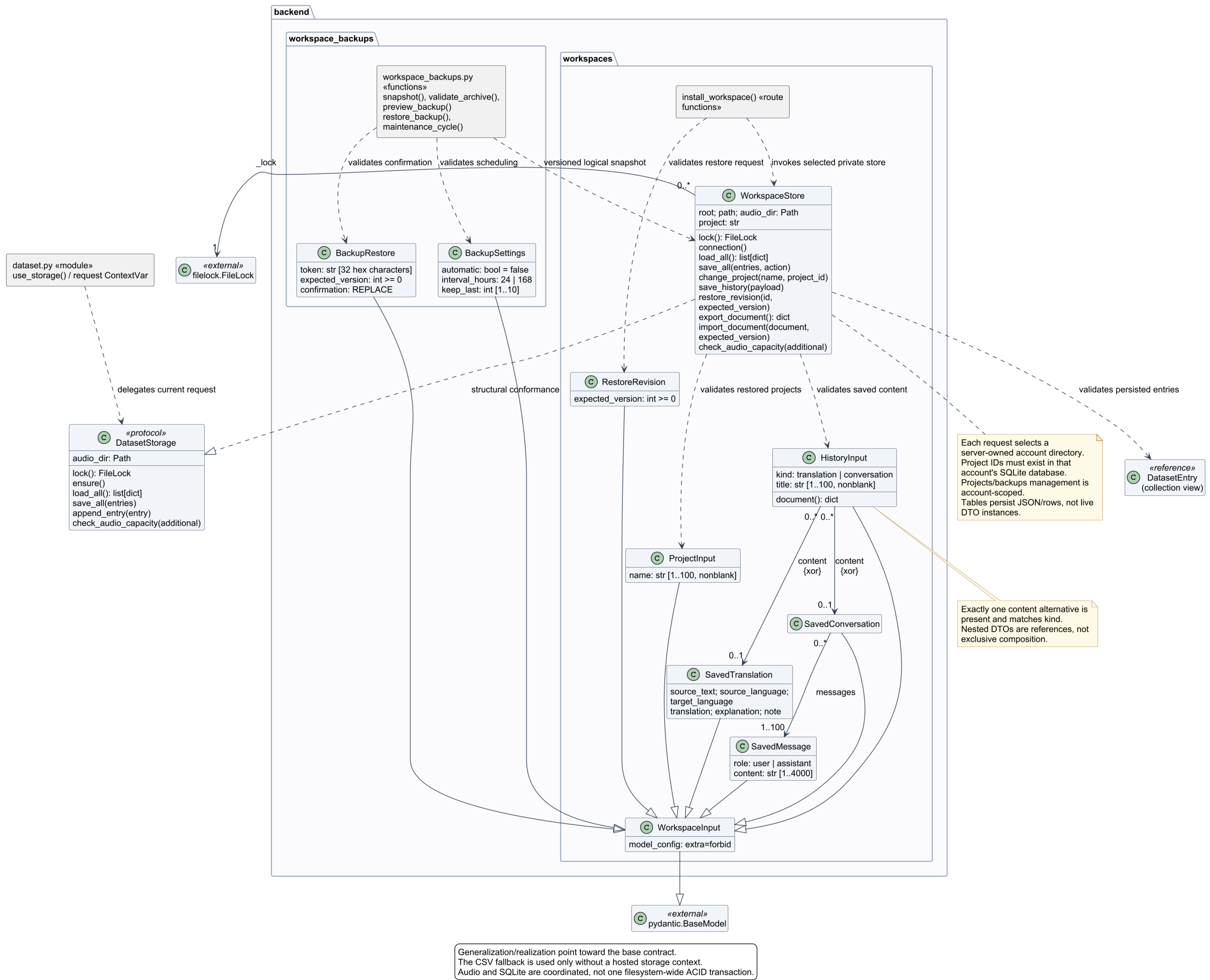


Hosted accounts: actual classes and selected operations



Hollow triangles: inheritance. Dashed arrows: dependencies, not object ownership. Associations describe actual stored references and include multiplicities. Secrets stay server-side; UserIdentity is the public account DTO.

Private projects, saved work and recovery: actual classes



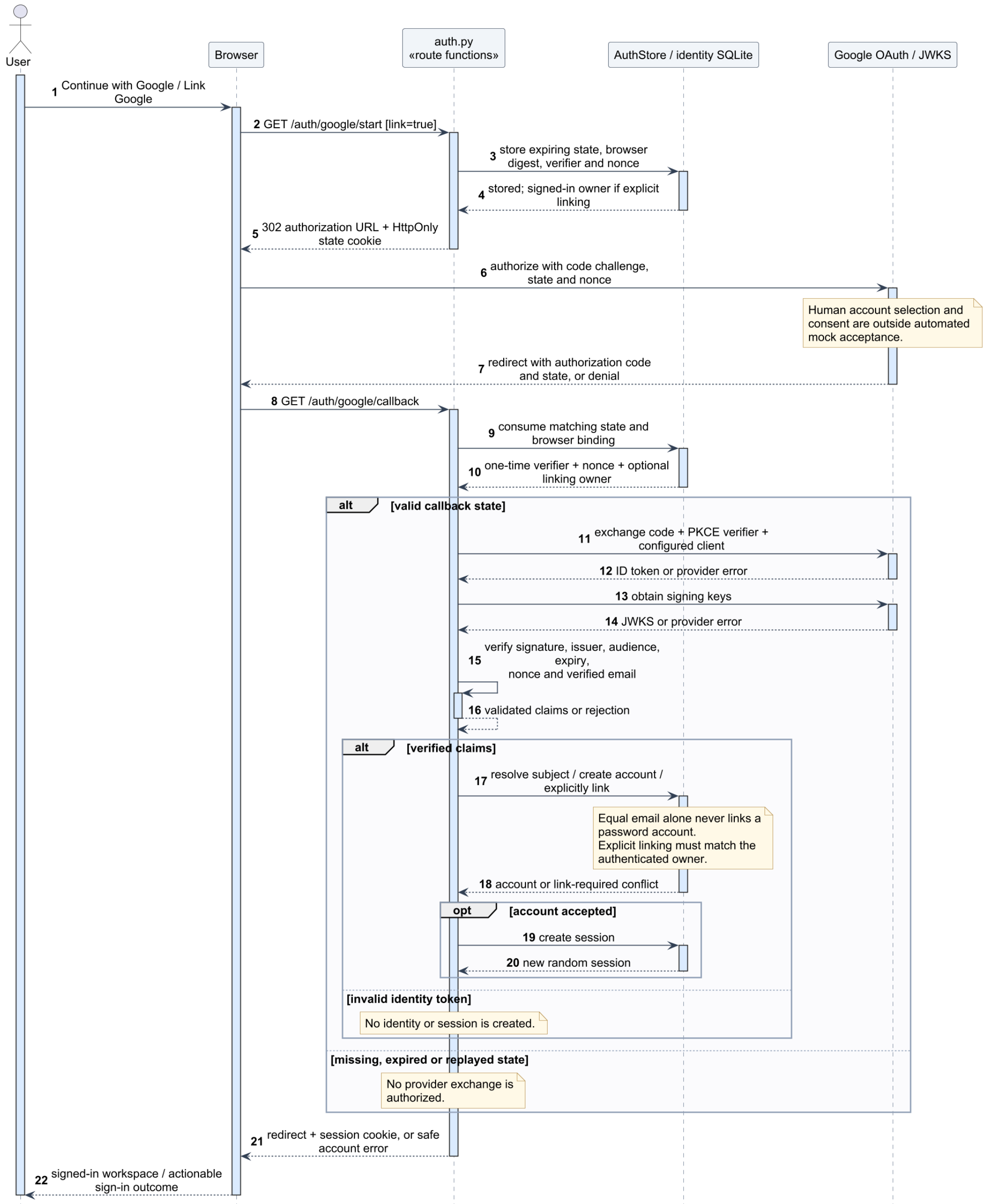
```

sequenceDiagram
    participant User
    participant AuthGate as AuthGate «React function»
    participant auth_py as auth.py «routes / middleware»
    participant store as store : AuthStore
    participant smtp as SMTP / development outbox
    participant workspaces_py as workspaces.py «context factory»
    participant workspace as workspace : WorkspaceStore
    participant private_api as Private API route

    User->>AuthGate: 1 submit registration
    activate AuthGate
    AuthGate->>auth_py: 2 POST /auth/register
    activate auth_py
    auth_py->>store: 3 throttle; create identity; issue verification token
    activate store
    store-->>auth_py: 4 eligible identity + one-time token, or no new identity
    deactivate store
    Note over auth_py: Password hashing and database work run off the event loop. Store only password hashes and token digests.
    auth_py->>smtp: 5 deliver verification link
    activate smtp
    smtp-->>auth_py: 6 accepted or explicit delivery error
    deactivate smtp
    auth_py-->>AuthGate: 7 generic 202 message or service error
    deactivate auth_py
    AuthGate-->>User: 8 check email
    User->>AuthGate: 9 open verification link and press Verify
    activate AuthGate
    AuthGate->>auth_py: 10 POST /auth/verify-email {token}
    activate auth_py
    auth_py->>store: 11 consume_token(token, verify)
    activate store
    store-->>auth_py: 12 verified or expired/used error
    deactivate store
    auth_py-->>AuthGate: 13 outcome
    deactivate auth_py
    AuthGate-->>User: 14 sign-in form or actionable error
    User->>AuthGate: 15 submit email and password
    activate AuthGate
    AuthGate->>auth_py: 16 POST /auth/login
    activate auth_py
    auth_py->>store: 17 verify password + verified status; create session
    activate store
    store-->>auth_py: 18 identity + random session, or failure
    deactivate store
    auth_py-->>AuthGate: 19 HttpOnly cookie + session-specific CSRF token
    deactivate auth_py
    AuthGate-->>User: 20 private workspace
    User->>AuthGate: 21 create terminology in selected project
    activate AuthGate
    AuthGate->>auth_py: POST /api/dataset  
22 cookie, Origin, CSRF, project header
    activate auth_py
    auth_py->>store: 23 validate session and enforce request allowance
    activate store
    store-->>auth_py: 24 verified identity or rejection
    deactivate store
    alt [valid session, origin and CSRF]
        auth_py->>workspaces_py: 25 enter async private workspace context
        activate workspaces_py
        workspaces_py->>workspace: 26 initialize/validate account and project off event loop
        activate workspace
        workspace-->>workspaces_py: 27 store or storage error
        deactivate workspace
        workspaces_py-->>auth_py: 28 request-local storage binding
        deactivate workspaces_py
        auth_py->>private_api: 29 dispatch validated request
        activate private_api
        private_api->>workspace: 30 locked transactional mutation
        activate workspace
        workspace-->>private_api: 31 saved entry + recorded revision, or failure
        deactivate workspace
        private_api-->>auth_py: 32 response
        deactivate private_api
        auth_py->>workspaces_py: 33 exit context
        activate workspaces_py
        workspaces_py-->>auth_py: 34 previous ContextVars restored
        deactivate workspaces_py
    else [access rejected]
        Note over auth_py: No private route executes; return 401 / 403 / 429.
    end
    auth_py-->>AuthGate: 35 result or explicit error
    deactivate auth_py
    AuthGate-->>User: 36 saved state or retained draft + error
    deactivate AuthGate
  
```

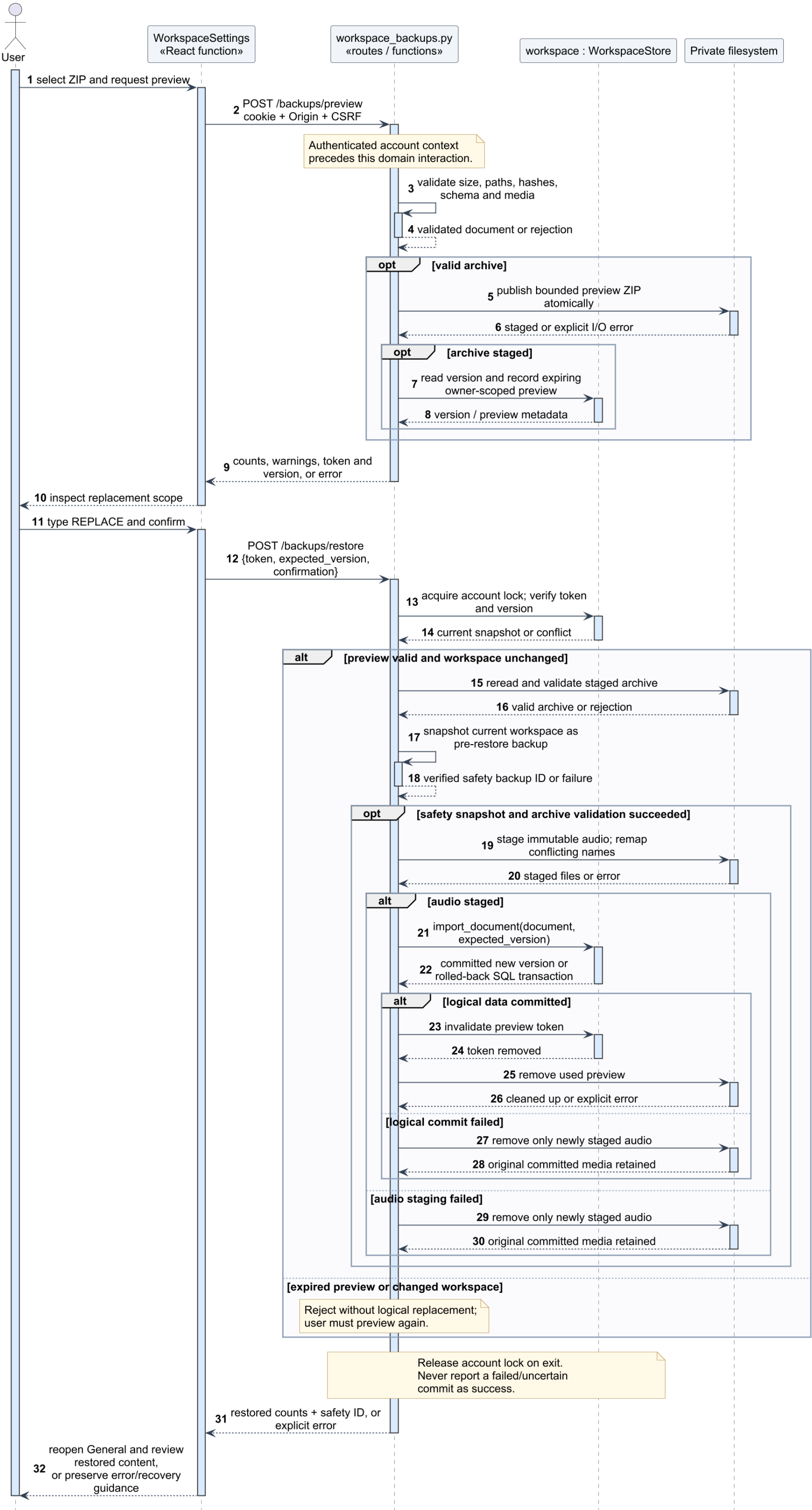
Registration, verification and login are separate interactions, not automatic sign-in.
Synchronous/awaited messages all have explicit activation boxes and matching replies.
Password reset uses the same single-use token boundary and revokes every prior session.

Google sign-in / explicit linking: authorization-code flow with PKCE



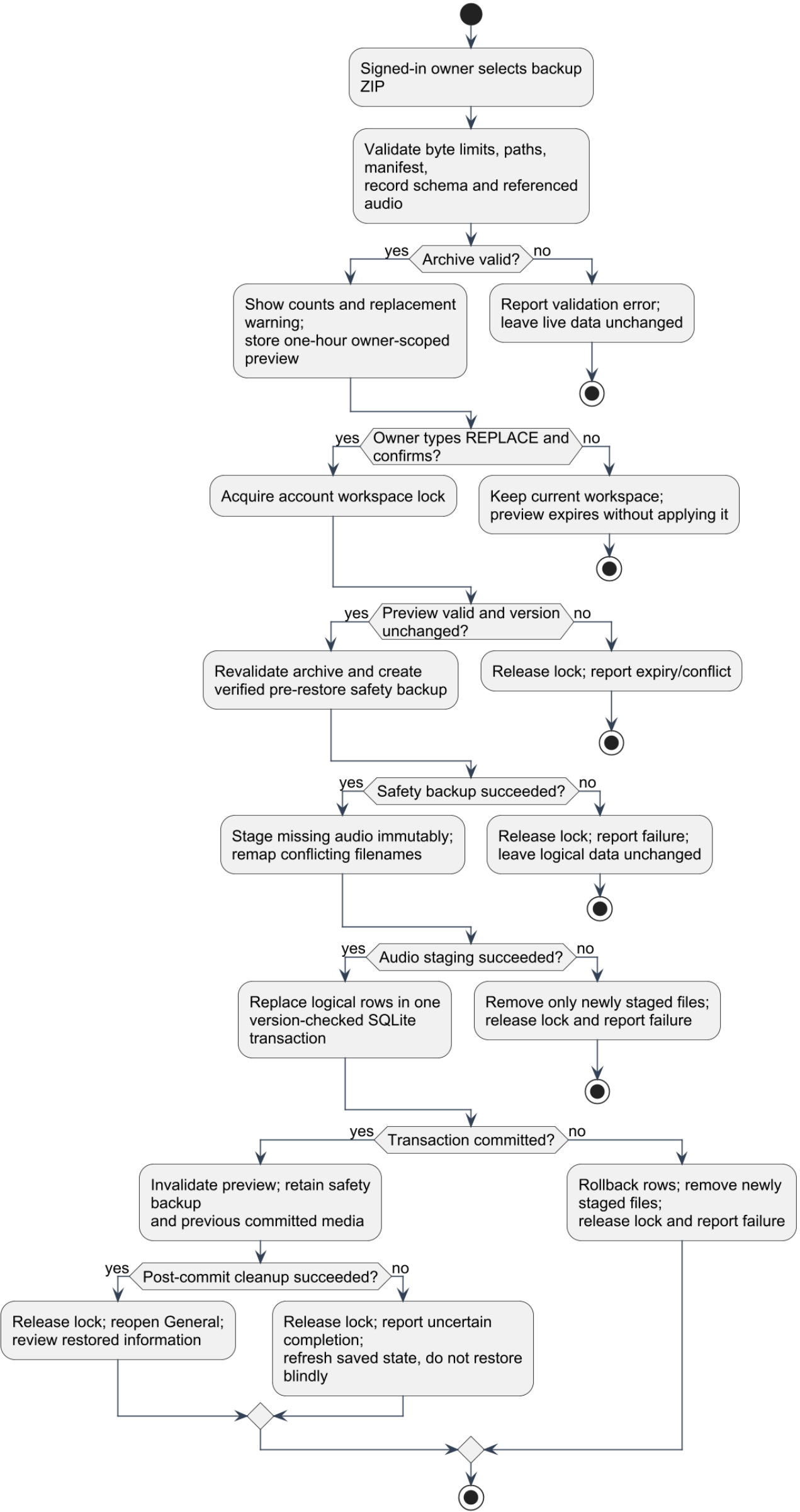
Redirects are HTTP replies; following them begins a new request.
Code, verifier, nonce and credentials are not written to browser storage.
Query values are redacted from application access logs.

Validated account backup restore with a safety copy



Backups replace logical workspace data, not account credentials or server settings.
Late client cancellation does not promise server rollback.
Existing committed media remains available for recovery; this is not a cross-file ACID transaction.

Private workspace restore: guarded replacement and failure paths



Branch guards are exclusive; each final node ends this attempt.
A cleanup error is explicit and does not imply a committed transaction rolled back.
This activity does not delete account credentials or other users' workspaces.